Cryptocurrency and Blockchain Technology

EE 465 — IIT Bombay

Prof. Saravanan Vijayakumaran
Department of Electrical Engineering
Indian Institute of Technology Bombay

2026

These lecture notes cover the complete EE 465 course. Topics span Bitcoin (transactions, mining, scripting), cryptographic foundations (hash functions, group theory, elliptic curve cryptography), Ethereum (accounts, EVM, smart contracts, Solidity, data structures), privacy mechanisms (Tornado Cash, zk-SNARKs), and consensus theory (Byzantine broadcast, Tendermint, Ethereum Proof-of-Stake).

Contents

1 Introduction to Bitcoin **9**

1.1 What Is Bitcoin? . . . 9

1.1.1 The Double-Spending Problem . . . 9

1.1.2 Network Participants . . . 9

1.2 Cryptographic Keys and Addresses . . . 9

1.2.1 Key Pair . . . 9

1.2.2 Bitcoin Address Derivation . . . 10

1.3 The Blockchain . . . 10

1.3.1 Block Header . . . 10

1.3.2 Merkle Hash Tree . . . 10

1.3.3 Tamper Resistance and the 51% Attack . . . 11

1.4 The Bitcoin P2P Network . . . 11

1.4.1 Node Discovery . . . 11

1.4.2 Transaction Propagation . . . 11

1.4.3 Block Propagation and Orphan Blocks . . . 12

1.4.4 Simplified Payment Verification (SPV) . . . 12

1.5 The Genesis Block . . . 13

1.6 The UTXO Set . . . 13

1.6.1 Key Takeaways . . . 13

2 Cryptographic Hash Functions **15**

2.1 Definition . . . 15

2.1.1 Security Hierarchy . . . 15

2.1.2 Attack Complexities . . . 15

2.2 SHA-256 . . . 16

2.2.1 Bitcoin Usage of SHA-256 . . . 16

2.3 SHA-256 Internal Structure in Detail . . . 16

2.3.1 Padding . . . 16

2.3.2 The Compression Function . . . 17

2.3.3 Avalanche Effect . . . 17

2.3.4 Length Extension Attacks . . . 17
2.4 RIPEMD-160 . . . 18
2.5 Hash Pointers . . . 18

3 Group Theory and Modular Arithmetic 19
3.1 Groups . . . 19
3.1.1 Examples Relevant to Cryptography . . . 19
3.2 Cyclic Groups and the Discrete Logarithm . . . 20
3.3 Modular Arithmetic . . . 20
3.4 Finite Fields . . . 20
3.4.1 The Extended Euclidean Algorithm . . . 20
3.4.2 Worked Example: Discrete Logarithm in $\mathbb{Z}_{11}^*$ . . . 21
3.5 Lagrange's Theorem . . . 21
3.6 Quadratic Residues . . . 21

4 Elliptic Curve Cryptography in Bitcoin 22
4.1 Elliptic Curves . . . 22
4.1.1 Bitcoin's Curve: secp256k1 . . . 22
4.2 Group Law on Elliptic Curves . . . 22
4.2.1 Point Addition: $P + Q$ with $P \neq \pm Q$ . . . 22
4.2.2 Point Doubling: $P + P = 2P$ . . . 23
4.2.3 Identity and Inverse . . . 23
4.3 Scalar Multiplication . . . 23
4.4 Worked Example: Point Addition over $\mathbb{F}_{17}$ . . . 23
4.5 Elliptic Curve Discrete Logarithm Problem (ECDLP) . . . 24
4.6 Key Generation . . . 24
4.7 ECDSA: Elliptic Curve Digital Signature Algorithm . . . 24
4.7.1 Signing . . . 24
4.7.2 Verification . . . 25
4.8 ECDSA Worked Conceptual Example . . . 25
4.8.1 Schnorr Signatures (Taproot) . . . 26
4.9 Parameter Summary . . . 26

5 Bitcoin Transactions 27
5.1 The UTXO Model . . . 27
5.2 Transaction Structure . . . 27
5.2.1 Input Fields . . . 28
5.2.2 Output Fields . . . 28
5.3 Bitcoin Script . . . 28

5.3.1 Key Opcodes . . . 28
5.4 P2PKH — Pay to Public Key Hash . . . 29
5.5 P2SH — Pay to Script Hash . . . 29
5.6 Multisignature (m-of-n) . . . 29
5.7 Worked Example: A Complete Bitcoin Transaction . . . 30
5.8 P2WPKH: Pay-to-Witness-Public-Key-Hash . . . 30
5.9 Timelock Transactions . . . 30
5.10 SegWit and Transaction Malleability . . . 31
5.11 Coinbase Transaction . . . 31

6 Mining Miscellanea **32**
6.1 Proof of Work . . . 32
6.1.1 Difficulty and Adjustment . . . 32
6.1.2 Expected Hashes per Block . . . 32
6.2 Mining Hardware . . . 32
6.3 Mining Pools . . . 33
6.4 Why SHA256d and Not SHA256? . . . 33
6.4.1 Nonce Space Exhaustion . . . 33
6.5 Selfish Mining . . . 33
6.6 Transaction Lifecycle in the Mempool . . . 33
6.7 The Lightning Network . . . 34
6.8 Proof-of-Work vs. Alternatives . . . 34

7 Introduction to Ethereum **35**
7.1 Ethereum vs. Bitcoin . . . 35
7.2 Account Model . . . 35
7.2.1 Externally Owned Accounts (EOAs) . . . 35
7.2.2 Contract Accounts . . . 35
7.3 Ether and Wei . . . 36
7.4 Gas and Fees . . . 36
7.4.1 Why Gas? . . . 36
7.4.2 EIP-1559 Fee Mechanism (Post-London 2021) . . . 36
7.4.3 EIP-1559: Worked Fee Example . . . 37
7.4.4 Account Nonce and Replay Protection . . . 37
7.5 Smart Contracts . . . 37
7.5.1 Representative Applications . . . 38
7.6 ERC-20 Token Standard . . . 38

8 Ethereum Data Structures and Encoding **39**

8.1 RLP Encoding . . . 39
8.1.1 Encoding Rules . . . 39
8.1.2 Examples . . . 39
8.2 Merkle Patricia Trie (MPT) . . . 40
8.2.1 Node Types . . . 40
8.2.2 Hex-Prefix (HP) Path Encoding . . . 40
8.3 MPT Worked Example . . . 40
8.4 The Four Ethereum Tries . . . 41
8.5 Keccak-256 . . . 41
8.6 Bloom Filters . . . 41

9 Ethereum Transactions 43
9.1 Transaction Types . . . 43
9.2 EIP-1559 Transaction Fields . . . 43
9.3 Signing and Sender Recovery . . . 44
9.4 Transaction Execution Flow . . . 44
9.5 Contract Creation . . . 44
9.6 Function Calls . . . 44
9.7 Internal Calls . . . 44

10 Ethereum Virtual Machine (EVM) 46
10.1 Architecture . . . 46
10.2 Instruction Set . . . 46
10.3 Execution Trace Example . . . 46
10.4 Gas Accounting in Practice . . . 46
10.4.1 Out-of-Gas and Reverts . . . 47
10.4.2 Call Stack and Call Depth Limit . . . 47
10.5 ABI Encoding . . . 48
10.6 Contract Deployment Bytecode . . . 48

11 Ethereum Blocks 49
11.1 Block Structure . . . 49
11.1.1 Execution Block Header . . . 49
11.2 Transaction Receipts . . . 49
11.3 Logs and Events . . . 50
11.4 The Merge (September 15, 2022) . . . 50

12 Solidity Programming 51
12.1 Language Overview . . . 51

12.2 Types . . . 51
12.2.1 Value Types . . . 51
12.2.2 Reference Types . . . 51
12.2.3 Special Variables . . . 51
12.3 Function Visibility and Mutability . . . 52
12.4 Mappings, Structs, Events . . . 52
12.5 Modifiers and Inheritance . . . 53
12.6 Minimal ERC-20 Implementation . . . 53
12.7 Storage Layout . . . 54
12.8 Interfaces and Abstract Contracts . . . 55
12.9 The Proxy Pattern (Upgradeable Contracts) . . . 55
12.10 Common Vulnerabilities . . . 56

13 JavaScript and Node.js **58**
13.1 JavaScript Essentials . . . 58
13.2 Arrays . . . 58
13.3 Objects . . . 59
13.4 Promises and Async/Await . . . 59
13.5 Node.js . . . 59
13.6 Accepting CLI Input with Inquirer . . . 60
13.7 Connecting to Ethereum with ethers.js . . . 60

14 Tornado Cash and Privacy **61**
14.1 Motivation . . . 61
14.2 Protocol Overview . . . 61
14.2.1 Desired Properties . . . 61
14.3 Commitment Scheme . . . 61
14.3.1 Private Note . . . 61
14.3.2 Pedersen Hash . . . 62
14.4 Deposit Steps . . . 62
14.5 Zero-Knowledge Proofs: zk-SNARKs . . . 62
14.5.1 What Is a zk-SNARK? . . . 62
14.5.2 The Withdrawal Statement . . . 62
14.6 Withdrawal Steps . . . 63
14.7 Groth16: The Proof System Used by Tornado Cash . . . 63
14.7.1 Arithmetic Circuits . . . 64
14.7.2 Quadratic Arithmetic Programs (QAP) . . . 64
14.7.3 Groth16 Proof and Verification . . . 64

14.7.4 Poseidon Hash . . . 64
14.7.5 Trusted Setup Ceremonies . . . 64
14.8 The Circom Circuit . . . 65
14.9 OFAC Sanctions . . . 65

15 Consensus Protocols **67**
15.1 State Machine Replication (SMR) . . . 67
15.2 Network Models . . . 67
15.3 Fault Types . . . 67
15.4 SMR with Rotating Leaders (Honest Setting) . . . 68
15.5 Byzantine Broadcast (BB) . . . 68
15.5.1 Protocol for $f = 1$ (Cross-Checking) . . . 68
15.6 The Dolev-Strong Protocol . . . 69
15.7 Correctness of Dolev-Strong . . . 69
15.8 Impossibility Results . . . 70
15.8.1 FLM (No-PKI Bound) . . . 70
15.8.2 FLP (Asynchronous Impossibility) . . . 70
15.8.3 Partially Synchronous Bound . . . 70
15.9 Consensus Summary Table . . . 70

16 Tendermint Protocol **72**
16.1 Overview . . . 72
16.2 Rounds and Phases . . . 72
16.3 Quorum Certificates . . . 72
16.4 Local State per Node . . . 73
16.5 Protocol Pseudocode . . . 73
16.6 Proof of Consistency . . . 74
16.7 Liveness in Tendermint (Post-GST) . . . 74
16.8 Safety vs. Liveness in Partially Synchronous Protocols . . . 75
16.9 Practical Deployment . . . 75

17 Ethereum Proof-of-Stake **76**
17.1 The Merge . . . 76
17.2 Node Architecture . . . 76
17.3 Validators . . . 76
17.4 Slots and Epochs . . . 76
17.5 Committees and Attestations . . . 77
17.6 BLS Signatures . . . 77
17.6.1 Mathematical Foundation . . . 77

17.6.2 Aggregation 78
17.7 Validator and Committee Selection 78
17.7.1 RANDAO: On-Chain Randomness 78
17.7.2 Committee Assignment Algorithm 78
17.8 LMD GHOST Fork Choice Rule 79
17.8.1 LMD GHOST Worked Example 79
17.9 Casper FFG 79
17.9.1 Checkpoints 80
17.9.2 FFG Votes 80
17.9.3 Justification 80
17.9.4 Finalization 80
17.10 Casper FFG Slashing Rules 80
17.10.1 Rule 1 — No Double Vote 80
17.10.2 Rule 2 — No Surround Vote 80
17.11 Accountable Safety 81
17.12 Ethereum PoS Parameter Summary 81

A Mathematical Quick Reference **82**
A.1 Group Theory 82
A.2 Elliptic Curve Operations 82
A.3 Hash Function Summary 82

B Glossary **84**

Chapter 1

Introduction to Bitcoin

1.1 What Is Bitcoin?

Bitcoin is a decentralised peer-to-peer electronic cash system introduced in 2008 by the pseudonymous **Satoshi Nakamoto**. The core innovation is solving the *double-spending problem* without a trusted central authority.

Key Insight

Bitcoin replaces institutional trust (a bank) with mathematical trust (cryptographic proof) enforced by a distributed network of nodes running identical validation rules.

1.1.1 The Double-Spending Problem

In physical cash Alice hands Bob a $20 note and can no longer spend it. Digital files can be copied trivially, so Alice could send the same digital coin to both Bob and Carol. Traditional systems rely on a bank to prevent this. Bitcoin prevents it through the **blockchain**: a public ledger whose ordering is agreed upon by *proof of work*.

1.1.2 Network Participants

Full node Downloads and validates every block and transaction.

Miner Bundles transactions into blocks and solves PoW.

Light client Stores only block headers; uses Merkle proofs.

Wallet Manages keys, creates and broadcasts transactions.

1.2 Cryptographic Keys and Addresses

1.2.1 Key Pair

- **Private key** k: uniformly random 256-bit integer in $[1, n-1]$.

- **Public key** $K = k \cdot G$: a point on the secp256k1 curve (Chapter 4).

1.2.2 Bitcoin Address Derivation

$$H_1 = \text{SHA256}(K) \tag{1.1}$$
$$H_2 = \text{RIPEMD160}(H_1) \quad \text{(20 bytes)} \tag{1.2}$$
$$\text{checksum} = \text{SHA256}(\text{SHA256}(\texttt{0x00} \| H_2))[0:4] \tag{1.3}$$
$$\text{address} = \text{Base58Check}(\texttt{0x00} \| H_2 \| \text{checksum}) \tag{1.4}$$

Exam Note

The address is a *hash* of the public key, not the public key itself. The public key is revealed only when spending. This gives limited post-quantum resistance.

1.3 The Blockchain

1.3.1 Block Header

Table 1.1: Bitcoin Block Header Fields (80 bytes total)

Field	Size	Description
`nVersion`	4 B	Protocol version
`hashPrevBlock`	32 B	SHA256d of previous block header
`hashMerkleRoot`	32 B	Merkle root of all transactions
`nTime`	4 B	Unix timestamp
`nBits`	4 B	Compact target encoding
`nNonce`	4 B	Variable miners iterate to meet target

Figure 1.1: Each block commits to its predecessor via a hash pointer.

1.3.2 Merkle Hash Tree

Transactions are hashed pairwise up a binary tree until a single *Merkle root* remains:

$$h_{00} = H(t_0), \quad h_{01} = H(t_1), \quad h_0 = H(h_{00} \| h_{01}) \tag{1.5}$$
$$h_{10} = H(t_2), \quad h_{11} = H(t_3), \quad h_1 = H(h_{10} \| h_{11}) \tag{1.6}$$
$$\text{root} = H(h_0 \| h_1) \tag{1.7}$$

If the transaction count is not a power of two, the last hash is duplicated to pad the tree.

Merkle proofs. A light client can verify that t_i is in a block using $O(\log n)$ sibling hashes, without downloading the full block.

1.3.3 Tamper Resistance and the 51% Attack

Modifying any transaction in block B_N changes $H(B_N)$, which breaks the hash pointer in B_{N+1}, forcing a re-mine of every subsequent block. An attacker must outpace all honest miners:

$$\text{Current Bitcoin hashrate} \approx 10^{21} \text{ H/s} \tag{1.8}$$

$$\text{Cost of 50\% attack} \approx \$7.5 \text{ billion in hardware} \tag{1.9}$$

Key Insight

Bitcoin's security is economic: attacking the network costs more than the potential gain, as long as no entity controls $\geq 50\%$ of the hashrate.

1.4 The Bitcoin P2P Network

Bitcoin has no central server. Every full node maintains a copy of the entire blockchain ($\approx$ 600 GB as of 2024) and communicates directly with other nodes.

1.4.1 Node Discovery

1. A new node starts by connecting to **DNS seeds** — hard-coded domain names maintained by the Bitcoin community that return IP addresses of well-connected nodes.
2. The node sends a `version` message to its peer, including its best block height.
3. The peer responds with `verack` and its own `version`.
4. They exchange `addr` messages to discover more peers.
5. A node typically maintains 8 outbound and up to 125 inbound connections.

1.4.2 Transaction Propagation

When a user broadcasts a transaction:

1. The wallet sends the transaction to a connected full node.
2. The node validates the transaction (correct signatures, UTXOs exist, fee $\geq$ minimum relay fee).

3. If valid, the node adds it to its **mempool** and relays an `inv` (inventory) message to its peers.
4. Peers that do not yet have the transaction respond with `getdata`.
5. The transaction is forwarded; it reaches most of the network within a few seconds.

1.4.3 Block Propagation and Orphan Blocks

When a miner finds a valid block:

1. It broadcasts the block immediately.
2. Other nodes validate (check PoW, check all transactions) and relay.
3. If two miners find valid blocks at the same height almost simultaneously, a temporary **fork** occurs.
4. Nodes follow the **longest chain rule** (more precisely, the chain with the most cumulative work).
5. The fork resolves when one branch gets the next block; nodes on the shorter branch switch to the longer chain.
6. Blocks on the abandoned branch are called **orphan blocks** (stale blocks). Their transactions return to the mempool.

Key Insight

The longest-chain rule is not just about block count — it is about cumulative proof-of-work (sum of difficulties). A single block with higher difficulty can outweigh two blocks with lower difficulty.

1.4.4 Simplified Payment Verification (SPV)

Light clients implement **SPV**: they download only block headers (80 bytes each) rather than full blocks.

- A block header contains the Merkle root of all transactions.
- To verify that transaction t_i is in block B, the SPV client requests a **Merkle proof**: the sibling hashes on the path from leaf t_i to the root.
- The client recomputes the root from t_i and the proof and checks it matches the `hashMerkleRoot` in the header.
- This requires $O(\log n)$ hashes instead of downloading all n transactions.
- SPV trusts that the chain with the most PoW contains valid transactions — it does not independently verify all transactions.

1.5 The Genesis Block

The **genesis block** (block 0) was mined by Satoshi Nakamoto on January 3, 2009. It is hard-coded into the Bitcoin software.

- The coinbase transaction contains the famous message: *"The Times 03/Jan/2009 Chancellor on brink of second bailout for banks."* — a newspaper headline proving it was not pre-mined before that date.
- Its `hashPrevBlock` is all zeros (no predecessor).
- The 50 BTC reward in the genesis block is unspendable due to an original quirk in the Bitcoin software.

1.6 The UTXO Set

At any point in time, the **UTXO set** is the complete collection of all unspent transaction outputs.

- Every full node maintains the UTXO set in memory (or on a fast database like LevelDB).
- As of 2024, the UTXO set contains $\approx$ 100 million UTXOs, requiring $\approx$ 5 GB of RAM.
- When a transaction is processed:
  1. Each input removes a UTXO from the set.
  2. Each output adds a new UTXO to the set.
- A transaction is valid only if all its inputs reference UTXOs currently in the UTXO set (and are properly signed).

Exam Note

Why not an account balance? The UTXO model enables parallel transaction validation (non-conflicting UTXOs can be validated simultaneously) and makes the state explicit: you can prove you have coins by pointing to specific UTXOs, without trusting a running total.

1.6.1 Key Takeaways

- Prevents double spending and tampering via blockchain + PoW.
- Secure only if no party controls $\geq 50\%$ of hashrate.
- Mining difficulty adjusts every 2016 blocks to target 10 min/block.
- Block subsidy halves every 210 000 blocks; total supply capped at 21 000 000 BTC.

- The P2P network propagates transactions and blocks without any central server.
- The UTXO model is the fundamental accounting primitive: no account balances, only unspent outputs.

Chapter 2

Cryptographic Hash Functions

2.1 Definition

Definition

A **cryptographic hash function** is a map $H : \{0,1\}^* \to \{0,1\}^n$ satisfying:

1. **Efficiency**: $H(x)$ is fast to compute for any x.
2. **Preimage resistance**: Given h, it is infeasible to find any x with $H(x) = h$.
3. **Second-preimage resistance**: Given x, it is infeasible to find $x' \neq x$ with $H(x') = H(x)$.
4. **Collision resistance**: It is infeasible to find any distinct pair $x \neq x'$ with $H(x) = H(x')$.

2.1.1 Security Hierarchy

Collision resistance $\Rightarrow$ Second-preimage resistance (roughly). The reverse does not hold in general.

2.1.2 Attack Complexities

Table 2.1: Attack complexities for an n-bit hash

Attack	**Complexity**	**SHA-256** ($n = 256$)
Brute-force preimage	$O(2^n)$	2^{256}
Birthday collision	$O(2^{n/2})$	2^{128}

Birthday paradox: In a group of $\approx 1.2\sqrt{N}$ people, there is a $> 50\%$ chance two share a birthday. For hash collisions, $\approx 2^{n/2}$ evaluations suffice.

2.2 SHA-256

SHA-256 (Secure Hash Algorithm, 256-bit output) is a **Merkle–Damgård** construction:

1. Pad message to a multiple of 512 bits.
2. Split into 512-bit blocks $M_1, \ldots, M_k$.
3. Iterate: $H_i = \text{compress}(H_{i-1}, M_i)$ with $H_0 =$ fixed IV.
4. Output H_k (256 bits).

The compression function performs 64 rounds of mixing using: bitwise operations (AND, OR, XOR, NOT), bit rotations/shifts, modular addition (mod 2^{32}), and a schedule of 64 derived 32-bit words.

2.2.1 Bitcoin Usage of SHA-256

Table 2.2: SHA-256 in Bitcoin

Purpose	**Operation**
Block hash / PoW	SHA256(SHA256(header))
Transaction ID	SHA256(SHA256(serialised tx))
Merkle tree nodes	SHA256(SHA256($L \| R$))
Address step 1	SHA256(pubkey)
Checksum	First 4 B of SHA256(SHA256(payload))

2.3 SHA-256 Internal Structure in Detail

2.3.1 Padding

Before hashing, the message M of bit-length L is padded to a multiple of 512 bits:

1. Append a single bit '1'.
2. Append '0' bits until the length $\equiv 448 \pmod{512}$.
3. Append L as a 64-bit big-endian integer.

Example: hashing the 3-byte string `"abc"` ($L = 24$ bits):

```
Message:   61 62 63
Padding:   80 00...00 00 00 00 00 00 00 00 18
                   (length = 24 = 0x18)
Total: 512 bits (one block)
```

2.3.2 The Compression Function

SHA-256 uses eight 32-bit state words $H_0, \ldots, H_7$ initialised to the fractional parts of the square roots of the first 8 primes:

$$H_0 = \texttt{6a09e667}, \quad H_1 = \texttt{bb67ae85}, \quad \ldots$$

For each 512-bit message block, 64 rounds are performed. Each round uses one of 64 constants K_i (derived from cube roots of the first 64 primes) and the **message schedule** $W_0, \ldots, W_{63}$.

Message schedule (W_i for $i \geq 16$):

$$W_i = \sigma_1(W_{i-2}) + W_{i-7} + \sigma_0(W_{i-15}) + W_{i-16}$$

where $\sigma_0(x) = \mathrm{ROTR}^7(x) \oplus \mathrm{ROTR}^{18}(x) \oplus \mathrm{SHR}^3(x)$ and σ_1 is similar with rotation amounts 17, 19, 10.

Round function: let a, b, c, d, e, f, g, h be the working variables (initialised from $H_0, \ldots, H_7$):

$$T_1 = h + \Sigma_1(e) + \mathrm{Ch}(e, f, g) + K_i + W_i \tag{2.1}$$

$$T_2 = \Sigma_0(a) + \mathrm{Maj}(a, b, c) \tag{2.2}$$

$$h \leftarrow g,\ g \leftarrow f,\ f \leftarrow e,\ e \leftarrow d + T_1 \tag{2.3}$$

$$d \leftarrow c,\ c \leftarrow b,\ b \leftarrow a,\ a \leftarrow T_1 + T_2 \tag{2.4}$$

$$\begin{aligned}
\mathrm{Ch}(x, y, z) &= (x \wedge y) \oplus (\neg x \wedge z) \\
\mathrm{Maj}(x, y, z) &= (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \\
\Sigma_0(x) &= \mathrm{ROTR}^2(x) \oplus \mathrm{ROTR}^{13}(x) \oplus \mathrm{ROTR}^{22}(x) \\
\Sigma_1(x) &= \mathrm{ROTR}^6(x) \oplus \mathrm{ROTR}^{11}(x) \oplus \mathrm{ROTR}^{25}(x)
\end{aligned}$$

After 64 rounds, the block output is added back to $H_0, \ldots, H_7$ modulo 2^{32}.

2.3.3 Avalanche Effect

A one-bit change in the input propagates through the round function to affect (roughly) half the output bits within a few rounds.

Example 2.1.

```
SHA256("abc")     = ba7816bf 8f01cfea 414140de 5dae2ec7
                    3b338c12 12133393 8e04d2eb 4bb6e54
SHA256("abd")     = 3909e2e0 5f6dfb8b 6a6e1a3a 7b1a0b69
                    e9e4f5c3 9b0e7aa4 f2c2e75b e8f5a9c2
```

One character changes (c→d); every output byte is completely different.

2.3.4 Length Extension Attacks

SHA-256's Merkle-Damgård construction is vulnerable to **length extension attacks**: given $H(m)$, an attacker can compute $H(m \| \text{padding} \| m')$ without knowing m.

This is why Bitcoin uses **SHA256d** = SHA-256(SHA-256(·)): applying SHA-256 twice breaks the Merkle-Damgård chain and eliminates length extension vulnerability.

2.4 RIPEMD-160

RIPEMD-160 produces 20-byte output. Combined with SHA-256, it compresses a public key hash to 20 bytes for use in Bitcoin addresses. Using two independent hash families gives defence-in-depth against weaknesses in either.

2.5 Hash Pointers

A **hash pointer** stores both a data pointer and $H(\text{data})$. Modifying the data invalidates the pointer. Bitcoin's blockchain is a linked list of hash pointers:

$$\boxed{\texttt{hashPrevBlock} = \text{SHA256d(previous header)}} \tag{2.5}$$

Chapter 3

Group Theory and Modular Arithmetic

3.1 Groups

Definition

A **group** $(G, \cdot)$ is a set G with a binary operation satisfying:

(i) **Closure**: $a \cdot b \in G$ for all $a, b \in G$.

(ii) **Associativity**: $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.

(iii) **Identity**: $\exists\, e$ s.t. $e \cdot a = a \cdot e = a$ for all a.

(iv) **Inverses**: $\forall\, a\ \exists\, a^{-1}$ s.t. $a \cdot a^{-1} = e$.

If also $a \cdot b = b \cdot a$ for all a, b, the group is **abelian**.

3.1.1 Examples Relevant to Cryptography

Table 3.1: Common cryptographic groups

Group	Operation	Order	Cyclic?
$(\mathbb{Z}_n, +)$	Addition mod n	n	Yes
$(\mathbb{Z}_p^*, \times)$	Multiplication mod prime p	$p - 1$	Yes
$E(\mathbb{F}_p)$	Point addition on curve	$\approx p$	Yes (usually)

3.2 Cyclic Groups and the Discrete Logarithm

Definition

G is **cyclic** if $\exists\, g \in G$ (a *generator*) such that $G = \{g^0, g^1, \ldots, g^{|G|-1}\}$.

Exponentiation (easy): given k, compute $h = g^k$ in $O(\log k)$ group operations via double-and-add.

Discrete Logarithm Problem (DLP): given $h = g^k$, find k. Believed hard for suitable groups.

3.3 Modular Arithmetic

$$a \equiv b \pmod{n} \iff n \mid (a - b)$$

Modular inverse: if $\gcd(a, n) = 1$ then $a^{-1} \bmod n$ exists, computable via the Extended Euclidean Algorithm.

Theorem 3.1 (Fermat's Little Theorem)**.** *For prime p and $\gcd(a, p) = 1$:*

$$\boxed{a^{p-1} \equiv 1 \pmod{p}} \implies a^{-1} \equiv a^{p-2} \pmod{p}$$

3.4 Finite Fields

Definition

$\mathbb{F}_p = \mathbb{Z}_p$ for prime p: the set $\{0, 1, \ldots, p-1\}$ with addition and multiplication modulo p. Every non-zero element has a multiplicative inverse.

Elliptic curves in Bitcoin are defined over $\mathbb{F}_p$ for a 256-bit prime p.

3.4.1 The Extended Euclidean Algorithm

To find $a^{-1} \bmod n$, use the Extended Euclidean Algorithm (EEA):

Example 3.1. Find $7^{-1} \bmod 11$:

Run $\gcd(7, 11)$:

$$11 = 1 \cdot 7 + 4, \quad 7 = 1 \cdot 4 + 3, \quad 4 = 1 \cdot 3 + 1, \quad 3 = 3 \cdot 1$$

Back-substitute:

$$1 = 4 - 1 \cdot 3 = 4 - 1 \cdot (7 - 4) = 2 \cdot 4 - 7 = 2 \cdot (11 - 7) - 7 = 2 \cdot 11 - 3 \cdot 7$$

So $-3 \cdot 7 \equiv 1 \pmod{11}$, i.e. $7^{-1} \equiv -3 \equiv 8 \pmod{11}$.

Verify: $7 \times 8 = 56 = 5 \times 11 + 1 \equiv 1 \pmod{11}$. ✓

3.4.2 Worked Example: Discrete Logarithm in $\mathbb{Z}_{11}^*$

$\mathbb{Z}_{11}^* = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ is a cyclic group of order 10.

Finding generators: g is a generator iff $\text{ord}(g) = 10$.

Table 3.2: Powers of $g = 2$ in $\mathbb{Z}_{11}^*$

2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}
2	4	8	5	10	9	7	3	6	1

$g = 2$ generates all of $\mathbb{Z}_{11}^*$, so it is a generator.

DLP example: Given $h = 7$, find k such that $2^k \equiv 7 \pmod{11}$. From the table: $2^7 = 7$, so $k = 7$.

In practice (with 256-bit groups), no efficient algorithm is known.

3.5 Lagrange's Theorem

Theorem 3.2. *For finite group G and subgroup $H \leq G$, $|H|$ divides $|G|$.*

Corollary: The order of any $g \in G$ divides $|G|$, so $g^{|G|} = e$.

3.6 Quadratic Residues

$a \in \mathbb{F}_p^*$ is a **quadratic residue (QR)** if $\exists\, x$ with $x^2 \equiv a$.

Theorem 3.3 (Euler's Criterion)**.**

$$a^{(p-1)/2} \equiv \begin{cases} 1 \pmod{p} & a \text{ is a QR} \\ -1 \pmod{p} & a \text{ is a QNR} \end{cases}$$

Exactly $(p-1)/2$ elements of $\mathbb{F}_p^*$ are QRs.

Chapter 4

Elliptic Curve Cryptography in Bitcoin

4.1 Elliptic Curves

Definition

An **elliptic curve** over $\mathbb{F}_p$ is the set of points $(x, y) \in \mathbb{F}_p^2$ satisfying

$$y^2 \equiv x^3 + ax + b \pmod{p}, \quad \Delta = -16(4a^3 + 27b^2) \neq 0, \tag{4.1}$$

together with a formal *point at infinity* $\mathcal{O}$.

4.1.1 Bitcoin's Curve: secp256k1

$a = 0$, $b = 7$:

$$\boxed{y^2 \equiv x^3 + 7 \pmod{p}} \tag{4.2}$$

$p = 2^{256} - 2^{32} - 977 = \texttt{FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F}_{16}$

Group order:

$n = \texttt{FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141}_{16}$

4.2 Group Law on Elliptic Curves

The set $E(\mathbb{F}_p)$ forms an abelian group under *point addition*.

4.2.1 Point Addition: $P + Q$ with $P \neq \pm Q$

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p} \tag{4.3}$$

$$x_3 = \lambda^2 - x_1 - x_2 \pmod{p} \tag{4.4}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p} \tag{4.5}$$

Geometrically: draw the line through P and Q, reflect the third intersection point over the x-axis.

4.2.2 Point Doubling: $P + P = 2P$

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod p \quad (a = 0 \text{ for secp256k1}) \tag{4.6}$$

$$x_3 = \lambda^2 - 2x_1 \pmod p \tag{4.7}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod p \tag{4.8}$$

4.2.3 Identity and Inverse

- $P + \mathcal{O} = P$ (identity is $\mathcal{O}$)
- $-P = (x, -y \bmod p)$; $P + (-P) = \mathcal{O}$

4.3 Scalar Multiplication

$$kP = \underbrace{P + P + \cdots + P}_{k}$$

Efficient via **double-and-add** in $O(\log k)$ operations:

```
R = O (point at infinity)
for bit b in binary(k) from MSB to LSB:
    R = 2R          (doubling)
    if b == 1:
        R = R + P   (addition)
return R
```

4.4 Worked Example: Point Addition over $\mathbb{F}_{17}$

Consider the curve $E : y^2 \equiv x^3 + 2x + 2 \pmod{17}$.

Example 4.1. Let $P = (5, 1)$ and $Q = (6, 3)$ (both on the curve; verify: $1^2 = 1 \equiv 5^3 + 2 \cdot 5 + 2 = 137 \equiv 1 \pmod{17}$ ✓).

Compute $P + Q$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} = \frac{3 - 1}{6 - 5} = \frac{2}{1} = 2$$

$$x_3 = 2^2 - 5 - 6 = 4 - 11 = -7 \equiv 10 \pmod{17}$$

$$y_3 = 2(5 - 10) - 1 = -10 - 1 = -11 \equiv 6 \pmod{17}$$

So $P + Q = (10, 6)$.

Example 4.2. Compute $2P$ with $P = (5, 1)$:

$$\begin{aligned}
\lambda &= \frac{3 \cdot 5^2 + 2}{2 \cdot 1} = \frac{77}{2} = \frac{9}{2} \pmod{17} = 9 \cdot 2^{-1} \pmod{17} \\
2^{-1} &\equiv 9 \pmod{17} \quad (\text{since } 2 \cdot 9 = 18 \equiv 1) \\
\lambda &= 9 \cdot 9 = 81 \equiv 13 \pmod{17} \\
x_3 &= 13^2 - 2 \cdot 5 = 169 - 10 = 159 \equiv 6 \pmod{17} \\
y_3 &= 13(5 - 6) - 1 = -13 - 1 = -14 \equiv 3 \pmod{17}
\end{aligned}$$

So $2P = (6, 3) = Q$. Hence $P + Q = 2P + P = 3P$, meaning $Q = 2P$.

Example 4.3. Scalar multiplication $4P$:

$$2P = (6, 3), \quad 4P = 2(2P) = 2(6, 3)$$

Compute $2(6, 3)$: $\lambda = (3 \cdot 36 + 2)/(2 \cdot 3) = 110/6$. $6^{-1} \equiv 3 \pmod{17}$ (since $6 \cdot 3 = 18 \equiv 1$). $\lambda = 110 \cdot 3 \bmod 17 = 330 \bmod 17 = 6$. $x_3 = 36 - 12 = 24 \equiv 7$, $y_3 = 6(6 - 7) - 3 = -6 - 3 = -9 \equiv 8 \pmod{17}$. So $4P = (7, 8)$.

The point at infinity $\mathcal{O}$ is the additive identity. The curve over $\mathbb{F}_{17}$ has 18 points (including $\mathcal{O}$), forming a cyclic group of order 18.

4.5 Elliptic Curve Discrete Logarithm Problem (ECDLP)

Definition

Given generator G and point $Q = kG$, the **ECDLP** is to find k.

Best known attacks (Pollard's ρ) require $O(\sqrt{n})$ operations. For secp256k1: $\sqrt{n} \approx 2^{128}$ — computationally infeasible.

4.6 Key Generation

$$k \xleftarrow{\$} \{1, \ldots, n-1\} \quad \text{(private key)} \tag{4.9}$$

$$\boxed{K} = \boxed{k \cdot G} \quad \text{(public key, 33 bytes compressed)} \tag{4.10}$$

Compressed public key: prefix `02` if y is even, `03` if y is odd, followed by x-coordinate (32 bytes).

4.7 ECDSA: Elliptic Curve Digital Signature Algorithm

4.7.1 Signing

Given message m, private key k, generator G, group order n:

1. $e = H(m)$ (hash of message)
2. Choose random nonce $r \in [1, n-1]$
3. $R = r \cdot G = (x_R, y_R)$; set $r_{\text{sig}} = x_R \bmod n$
4. $s = r^{-1}(e + k \cdot r_{\text{sig}}) \bmod n$
5. Signature: (r_{sig}, s)

$$\boxed{s = r^{-1}\big(H(m) + k \cdot r_{\text{sig}}\big) \bmod n} \tag{4.11}$$

4.7.2 Verification

Given (r_{sig}, s), public key K:

1. $e = H(m)$
2. $u_1 = s^{-1}e \bmod n, \quad u_2 = s^{-1}r_{\text{sig}} \bmod n$
3. $R' = u_1 G + u_2 K$
4. Accept iff $R'_x \equiv r_{\text{sig}} \pmod{n}$

Security Warning

Never reuse the nonce r! If r is reused for two messages m_1, m_2, the private key can be recovered: $k = (s_1 - s_2)^{-1}(H(m_1) - H(m_2)) \bmod n$. This caused the PlayStation 3 key leak (2010).

4.8 ECDSA Worked Conceptual Example

Example 4.4. Why the verification equation works.

We have signature (r_{sig}, s) where $s = r^{-1}(e + k \cdot r_{\text{sig}}) \bmod n$.

During verification we compute $R' = u_1 G + u_2 K$ where $u_1 = s^{-1}e \bmod n$ and $u_2 = s^{-1}r_{\text{sig}} \bmod n$.

$$\begin{aligned} R' &= u_1 G + u_2 K = s^{-1}e \cdot G + s^{-1}r_{\text{sig}} \cdot kG \\ &= s^{-1}(e + k \cdot r_{\text{sig}}) \cdot G = s^{-1} \cdot sr \cdot G = r \cdot G = R \end{aligned}$$

So $R'_x = R_x = r_{\text{sig}}$. The equation works because only someone knowing k can produce an s that satisfies it.

4.8.1 Schnorr Signatures (Taproot)

Bitcoin's Taproot upgrade (BIP 340, activated November 2021) added **Schnorr signatures** as an alternative to ECDSA:

Signing: Choose random nonce r, compute $R = r \cdot G$.

$$s = r + H(R\|K\|m) \cdot k \pmod{n}$$

Signature: (R, s).

Verification: Check $s \cdot G = R + H(R\|K\|m) \cdot K$.

Advantages over ECDSA:

- **Linearity**: n Schnorr signatures on the same message can be aggregated into a single signature (MuSig). This enables efficient n-of-n multisig and reduces blockchain footprint.
- **Simpler security proof**: Provably secure in the random oracle model via a tighter reduction.
- **Batch verification**: n signatures can be verified faster than n independent verifications.

4.9 Parameter Summary

Table 4.1: secp256k1 at a glance

Curve	$y^2 = x^3 + 7 \pmod{p}$
Field prime p	$2^{256} - 2^{32} - 977$ (256 bits)
Group order n	$\approx 2^{256}$
Security level	≈ 128 bits
Private key	32 bytes (256 bits)
Compressed public key	33 bytes
Uncompressed public key	65 bytes
Signature (DER)	71–72 bytes

Chapter 5

Bitcoin Transactions

5.1 The UTXO Model

Definition

A **UTXO** (Unspent Transaction Output) is an output of a previous transaction that has not yet been spent. A wallet's *balance* is the sum of all UTXOs locked to its addresses.

Unlike the account model (Ethereum), Bitcoin has no notion of a running balance. Every transaction explicitly consumes UTXOs as inputs and creates new UTXOs as outputs.

Key Insight

A UTXO is like a physical coin: you spend it whole. If you pay 0.5 BTC from a 1 BTC UTXO, you create a 0.5 BTC output to the recipient and a change output back to yourself. The difference is the miner fee.

5.2 Transaction Structure

Table 5.1: Bitcoin Transaction Fields

Field	Size	Description
version	4 B	Format version
vin_count	varint	Number of inputs
vin[]	variable	Array of inputs
vout_count	varint	Number of outputs
vout[]	variable	Array of outputs
locktime	4 B	Earliest block/time for inclusion

Table 5.2: Transaction Input Fields

Field	Size	Description
txid	32 B	TXID of the transaction being spent
vout	4 B	Index of the output within that tx
scriptSig	variable	Unlocking script
sequence	4 B	RBF / relative locktime

5.2.1 Input Fields

5.2.2 Output Fields

Table 5.3: Transaction Output Fields

Field	Size	Description
value	8 B	Amount in satoshis (10^8 sat = 1 BTC)
scriptPubKey	variable	Locking script

Implicit fee:

$$\text{fee} = \sum \text{input values} - \sum \text{output values}$$

5.3 Bitcoin Script

Bitcoin uses a **stack-based** scripting language.

- **scriptPubKey** (locking script): conditions to spend the output.
- **scriptSig** (unlocking script): data that satisfies conditions.

A UTXO is spendable iff `scriptSig` $\|$ `scriptPubKey` leaves `TRUE` on the stack.

5.3.1 Key Opcodes

Table 5.4: Common Bitcoin Script opcodes

Opcode	Effect
`OP_DUP`	Duplicate top stack item
`OP_HASH160`	Apply SHA256 then RIPEMD160
`OP_EQUAL`	Pop two items; push 1 if equal
`OP_EQUALVERIFY`	Like EQUAL but fail if not equal
`OP_CHECKSIG`	Verify ECDSA signature
`OP_CHECKMULTISIG`	Verify m-of-n multisignature
`OP_RETURN`	Mark output unspendable (data)

5.4 P2PKH — Pay to Public Key Hash

Most common transaction type; locks output to a Bitcoin address.

scriptPubKey:

```
OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

scriptSig:

```
<Signature> <PublicKey>
```

Execution trace:

1. Push `<Sig>`, `<PubKey>`.
2. `OP_DUP`: duplicate `<PubKey>`.
3. `OP_HASH160`: hash the copy → `<PubKeyHash'>`.
4. Push `<PubKeyHash>` from scriptPubKey.
5. `OP_EQUALVERIFY`: assert `Hash'` = `Hash`.
6. `OP_CHECKSIG`: verify ECDSA signature. Stack: `[TRUE]`.

5.5 P2SH — Pay to Script Hash

Allows complex conditions (multisig, timelocks) to be encoded in a *redeemScript*; the locking script only contains the hash.

scriptPubKey:

```
OP_HASH160 <RedeemScriptHash> OP_EQUAL
```

scriptSig:

```
<Data...> <RedeemScript>
```

5.6 Multisignature (m-of-n)

```
// 2-of-3 redeemScript:
OP_2 <PubKey1> <PubKey2> <PubKey3> OP_3 OP_CHECKMULTISIG

// Unlocking:
OP_0 <Sig1> <Sig2>
```

`OP_0` compensates for a historical off-by-one bug in `OP_CHECKMULTISIG`.

5.7 Worked Example: A Complete Bitcoin Transaction

Example 5.1. Scenario: Alice wants to pay Bob 0.3 BTC. Alice has two UTXOs:

- UTXO A: 0.4 BTC from transaction `txid = aaa...`, output 0
- UTXO B: 0.2 BTC from transaction `txid = bbb...`, output 1

She decides to spend UTXO A only (0.4 BTC > 0.3 BTC payment).

Transaction structure:

- **Input**: UTXO A (`aaa...`:0) with Alice's signature and public key in `scriptSig`.
- **Output 0**: 0.3 BTC locked to Bob's address (P2PKH: `OP_DUP OP_HASH160 <Bob_hash> OP_EQUALVERIFY OP_CHECKSIG`).
- **Output 1**: 0.095 BTC locked to Alice's change address.
- **Implicit fee**: $0.4 - 0.3 - 0.095 = 0.005$ BTC paid to miner.

What happens to Alice's UTXO set?

- UTXO A (0.4 BTC) is *removed* from the set (spent).
- Two new UTXOs are *added*: Bob's 0.3 BTC and Alice's change 0.095 BTC.
- UTXO B (0.2 BTC) is unaffected.

5.8 P2WPKH: Pay-to-Witness-Public-Key-Hash

The SegWit equivalent of P2PKH. The `scriptPubKey` is:

```
OP_0 <20-byte-pubkey-hash>
```

The witness data (signature + public key) is stored in the `witness` field, outside the transaction body used to compute the TXID. This makes the TXID immutable with respect to signature changes (fixing transaction malleability).

5.9 Timelock Transactions

Absolute timelocks: `locktime` field specifies the earliest block height or timestamp when the transaction can be mined.

Relative timelocks (BIP 68): the `sequence` field specifies a minimum number of blocks or time since the referenced UTXO was created.

OP_CHECKLOCKTIMEVERIFY (OP_CLTV): a script opcode that fails if the current block height/time is less than the value on the stack.

Use case: Hash Time-Locked Contracts (HTLCs) — the basis of the Lightning Network payment channels. Alice pays Bob only if Bob reveals a secret s such that $H(s) = h$ before a deadline.

```
// HTLC locking script:
OP_IF
    OP_SHA256 <h> OP_EQUALVERIFY OP_DUP OP_HASH160 <Bob_hash>
OP_ELSE
    <deadline> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_DUP OP_HASH160
       <Alice_hash>
OP_ENDIF
OP_EQUALVERIFY OP_CHECKSIG
```

5.10 SegWit and Transaction Malleability

SegWit (BIP 141, activated August 2017) moves witness data (signatures) outside the transaction ID hash:

- Fixes **transaction malleability** (TXID could be changed by modifying signatures).
- Witness data discounted 75% for fee calculation.
- Enables the **Lightning Network**.

5.11 Coinbase Transaction

Every block's first transaction is the **coinbase**:

- Has no real inputs (creates new coins ex nihilo).
- Pays miner: block subsidy + sum of all transaction fees.
- Can embed arbitrary data in a special coinbase field.

Block subsidy schedule (halving every 210 000 blocks):

$$\text{Total supply} = \sum_{k=0}^{\infty} 210000 \times \frac{50}{2^k} = 21{,}000{,}000 \text{ BTC}$$

Chapter 6

Mining Miscellanea

6.1 Proof of Work

$$\boxed{\text{SHA256d(block header)} < T} \tag{6.1}$$

Miners increment `nNonce` (and extra nonce in the coinbase) until the block hash falls below target T.

6.1.1 Difficulty and Adjustment

$$D = \frac{T_{\text{genesis}}}{T_{\text{current}}}$$

Every 2016 blocks ($\approx$ 2 weeks) the target adjusts:

$$T_{\text{new}} = T_{\text{old}} \times \frac{\text{actual time for 2016 blocks}}{2016 \times 10\,\text{min}} \quad \text{(capped at factor of 4)}$$

6.1.2 Expected Hashes per Block

If each hash independently satisfies the target with probability $p = T/2^{256}$, the expected number of hashes is $1/p$.

6.2 Mining Hardware

Table 6.1: Mining hardware evolution

Era	**Hardware**	**Typical rate**	**Efficiency**
2009–10	CPU	10 MH/s	Poor
2010–11	GPU	100 MH/s	Better
2011–13	FPGA	1 GH/s	Good
2013+	ASIC	>100 TH/s	Best

A modern Bitmain Antminer S19 XP delivers ≈140 TH/s at 3010 W.

6.3 Mining Pools

Solo mining variance is very high (expected time per solo miner: years). **Mining pools** aggregate hash power:

- Pool sends members *share targets* (easier than block target).
- Members submit shares as proof of work.
- When pool finds a block, reward is split by shares.
- Common payout schemes: PPS (Pay-Per-Share), PPLNS.

6.4 Why SHA256d and Not SHA256?

Bitcoin uses **double SHA-256** (`SHA256d`) for block hashing:

$$\text{block hash} = \text{SHA256}(\text{SHA256}(\text{header}))$$

Reason 1 — Length extension resistance: SHA-256 has a length extension vulnerability. Applying it twice breaks the Merkle-Damgård chain.

Reason 2 — Extra security margin: Even if SHA-256 were reduced to a 128-bit effective security level, SHA256d provides an additional layer.

6.4.1 Nonce Space Exhaustion

The 4-byte `nNonce` field gives only $2^{32} \approx 4$ billion values. At modern hashrates ($> 10^{18}$ H/s), a miner exhausts the entire nonce space in microseconds.

Solution: Vary the **extra nonce** in the coinbase transaction (first input's `scriptSig` can hold up to 100 bytes). Changing the extra nonce changes the coinbase transaction, which changes the Merkle root, which changes the block header — giving the miner a fresh set of 2^{32} nonces to try.

In practice: modern mining hardware also varies the `nTime` field (within ± 2 hours of real time) for even more entropy.

6.5 Selfish Mining

A miner with $> 1/3$ of hashrate can mine a private chain, release it strategically, and earn revenue disproportionate to their share.

6.6 Transaction Lifecycle in the Mempool

1. User broadcasts transaction; nodes validate and add to **mempool**.
2. Miner selects transactions (highest fee/vbyte first) into a block.

3. Miner solves PoW; broadcasts block.
4. After ≈6 confirmations (≈60 min) the transaction is considered irreversible.

6.7 The Lightning Network

Second-layer protocol built on Bitcoin for fast, cheap payments:

1. Two parties open a **payment channel** — a 2-of-2 multisig UTXO.
2. They exchange signed off-chain transactions (channel updates).
3. Either party can close by broadcasting the latest signed state.
4. Payments route through a network of channels (no direct channel needed).

6.8 Proof-of-Work vs. Alternatives

Table 6.2: Consensus mechanism comparison

Mechanism	**Resource**	**Attack threshold**	**Examples**
Proof of Work	Compute / energy	50% hashrate	Bitcoin
Proof of Stake	Staked coins	33% of stake (BFT)	Ethereum, Cardano
DPoS	Elected delegates	Delegate collusion	EOS, TRON
Proof of Space	Disk storage	51% storage	Chia

Chapter 7

Introduction to Ethereum

7.1 Ethereum vs. Bitcoin

Table 7.1: Bitcoin vs. Ethereum

Feature	**Bitcoin**	**Ethereum**
Primary purpose	Digital currency	Programmable platform
State model	UTXO	Account-based
Scripting	Limited (Script)	Turing-complete (EVM)
Block time	≈10 min	12 s (post-merge)
Consensus	PoW	PoS (since Sept 2022)
Supply cap	21M BTC	None (issuance → 0)
Creator	Satoshi Nakamoto (2008)	Vitalik Buterin (2015)

7.2 Account Model

7.2.1 Externally Owned Accounts (EOAs)

- Controlled by a private key (human user).
- Can initiate transactions.
- Fields: `nonce`, `balance`.
- No code or storage.

7.2.2 Contract Accounts

- Controlled by immutable EVM code.
- Created by a deployment transaction.
- Fields: `nonce`, `balance`, `storageRoot`, `codeHash`.

- Executes when called by EOA or another contract.

$$\text{Account state} = (\underbrace{\texttt{nonce}}_{\text{tx count}}, \underbrace{\texttt{balance}}_{\text{wei}}, \underbrace{\texttt{storageRoot}}_{\text{trie hash}}, \underbrace{\texttt{codeHash}}_{\text{keccak256(code)}}) \tag{7.1}$$

7.3 Ether and Wei

$$1 \text{ ETH} = 10^{18} \text{ wei} \tag{7.2}$$
$$1 \text{ ETH} = 10^{9} \text{ gwei} \tag{7.3}$$

Gas prices are quoted in **gwei**.

7.4 Gas and Fees

Gas is the unit of computational work inside the EVM.

7.4.1 Why Gas?

- Prevents infinite loops (Turing halting problem) — execution stops when gas exhausted.
- Compensates validators for computation.
- Creates a market for block space.

7.4.2 EIP-1559 Fee Mechanism (Post-London 2021)

$$\text{effective gas price} = \min(\texttt{maxFeePerGas}, \texttt{baseFee} + \texttt{priorityFee}) \tag{7.4}$$
$$\text{fee paid} = \text{gasUsed} \times \text{effective gas price} \tag{7.5}$$

- **Base fee**: Algorithmically set; *burned* each block.
- **Priority fee (tip)**: Given to the validator.
- Base fee rises 12.5% if previous block was full; falls 12.5% if empty (target: 50% full).

7.4.3 EIP-1559: Worked Fee Example

Example 7.1. Given: base fee = 20 gwei, user sets `maxPriorityFeePerGas` = 2 gwei, `maxFeePerGas` = 25 gwei. Transaction uses 21 000 gas (simple ETH transfer).

Step 1: Effective gas price = $\min(25, 20 + 2) = 22$ gwei.

Step 2: Fee paid = $21000 \times 22 = 462000$ gwei = 0.000462 ETH.

Step 3: Base fee burned = $21000 \times 20 = 420000$ gwei.

Step 4: Validator tip = $21000 \times 2 = 42000$ gwei.

Step 5: Refund to user = $(25 - 22) \times 21000 = 63000$ gwei.

Summary: User paid 0.000462 ETH; 0.000420 ETH was burned; validator received 0.000042 ETH.

Key Insight

Deflationary effect: Base fee is *burned* (removed from circulation) rather than given to validators. During periods of high activity, ETH supply decreases. In 2021–2022 Ethereum was briefly *deflationary* (more ETH burned than issued).

7.4.4 Account Nonce and Replay Protection

Every transaction from an EOA includes a **nonce** equal to the number of transactions previously sent from that address.

- Prevents **replay attacks**: if Alice sends a transaction, an attacker cannot replay the same transaction (the nonce would be wrong).
- Enforces **transaction ordering**: transactions from the same sender are executed in nonce order.
- If a nonce is skipped (e.g., nonce 3 but nonce 2 not yet mined), the transaction is held in the mempool until its predecessor arrives.

EIP-155 replay protection: The transaction includes `chainId` in its signature hash. A transaction signed for Ethereum mainnet (chainId=1) cannot be replayed on Ethereum Classic (chainId=61) or any other chain.

7.5 Smart Contracts

Definition

A **smart contract** is a program deployed on the blockchain that executes deterministically on the EVM, has its own address/balance/ storage, is immutable once deployed, and is triggered by transactions.

7.5.1 Representative Applications

DeFi Uniswap (DEX), Aave/Compound (lending), MakerDAO (stablecoins).

NFTs ERC-721 unique digital assets.

DAOs On-chain governance (voting, treasury management).

Privacy Tornado Cash (Chapter 14).

7.6 ERC-20 Token Standard

Standard interface for fungible tokens:

- `totalSupply()` — total token supply.
- `balanceOf(address)` — balance of an address.
- `transfer(to, amount)` — transfer tokens.
- `approve(spender, amount)` — allow delegation.
- `transferFrom(from, to, amount)` — spend delegated tokens.
- Events: `Transfer`, `Approval`.

Chapter 8

Ethereum Data Structures and Encoding

8.1 RLP Encoding

Recursive Length Prefix (RLP) is the canonical serialisation format in Ethereum for all data structures.

8.1.1 Encoding Rules

Single byte b:

- $b \in [0x00, 0x7f]$: encode as b itself.

Byte string of length ℓ:

- $\ell = 0$: `0x80`.
- $\ell \leq 55$: $(0x80 + \ell) \parallel$ string.
- $\ell > 55$: $(0xb7 + \text{len}(\ell)) \parallel \text{BE}(\ell) \parallel$ string.

List with total payload length ℓ:

- $\ell \leq 55$: $(0xc0 + \ell) \parallel$ RLP-encoded items.
- $\ell > 55$: $(0xf7 + \text{len}(\ell)) \parallel \text{BE}(\ell) \parallel$ items.

8.1.2 Examples

- `RLP("dog")` = `0x83 64 6f 67` $(\ell = 3,\ 0x80 + 3 = 0x83)$
- `RLP(["cat","dog"])` = `0xc8 83 63 61 74 83 64 6f 67` (payload = 8 bytes, $0xc0 + 8 = 0xc8$)
- `RLP("")` = `0x80`
- `RLP([])` = `0xc0`

8.2 Merkle Patricia Trie (MPT)

Ethereum stores its state in a **Merkle Patricia Trie**: a compressed radix trie where every node is identified by its hash.

8.2.1 Node Types

Empty node Null — represents missing key.

Leaf node (k_{enc}, value) — stores a key-value pair.

Extension node (k_{enc}, hash_of_next) — compresses a shared key prefix.

Branch node 17 children: indices 0–f (hex nibble) plus an optional value slot.

8.2.2 Hex-Prefix (HP) Path Encoding

Keys are sequences of hex nibbles; a flag byte encodes type and parity:

Prefix nibble	Node type	Path length
`0`	Extension	Even
`1`	Extension	Odd
`2`	Leaf	Even
`3`	Leaf	Odd

8.3 MPT Worked Example

Example 8.1. Build a Merkle Patricia Trie with two key-value pairs:

- Key `0x1234` → value "dog"
- Key `0x1235` → value "cat"

Both keys share the nibble prefix `1,2,3`.

Step 1: Both paths share prefix 1,2,3. Create an **extension node** with key `1,2,3` pointing to a branch.

Step 2: After the shared prefix, the paths diverge at nibble 4 vs 5. Create a **branch node** with children at positions 4 and 5.

Step 3: Each child is a **leaf node**:

- Position 4: leaf (remaining path = empty, value = "dog")
- Position 5: leaf (remaining path = empty, value = "cat")

Resulting structure:

```
Extension(path=[1,2,3], next=Branch)
  Branch[4] -> Leaf(path=[], value="dog")
  Branch[5] -> Leaf(path=[], value="cat")
```

Merkle hashing: Each node is hashed with keccak256. The root hash is the hash of the extension node. Changing "dog" to "duck" changes the leaf hash, the branch hash, and the extension (root) hash. This is the Merkle property.

8.4 The Four Ethereum Tries

Table 8.1: Tries referenced in block headers

Trie	Key	Value (RLP-encoded)
State	`keccak256(address)`	(nonce, balance, storageRoot, codeHash)
Transactions	`RLP(tx index)`	serialised transaction
Receipts	`RLP(tx index)`	(status, gasUsed, logs, bloom)
Storage (per contract)	`keccak256(slot)`	256-bit value

Key Insight

The `stateRoot` in the block header is the root hash of the global state trie. Changing any account (balance, nonce, storage) changes the root, making tampering detectable.

8.5 Keccak-256

Ethereum uses **Keccak-256** (a variant of SHA-3) for:

- Address derivation: last 20 bytes of keccak256(uncompressed pubkey).
- EVM `KECCAK256` opcode.
- Function selectors: first 4 bytes of keccak256(signature string).
- All MPT node hashes.

8.6 Bloom Filters

Each block header contains a 2048-bit **Bloom filter**:

- Compact representation of all addresses and topics in the block's logs.
- False positives possible; false negatives impossible.

- Enables efficient log searching without a full node.

Chapter 9

Ethereum Transactions

9.1 Transaction Types

Type 0 (Legacy) Single `gasPrice` field; pre-EIP-1559.

Type 1 (EIP-2930) Adds `accessList` for storage slot pre-warming.

Type 2 (EIP-1559) `maxFeePerGas` + `maxPriorityFeePerGas`; dominant format today.

9.2 EIP-1559 Transaction Fields

Table 9.1: Type-2 transaction fields

Field	Type	Description
`chainId`	uint256	Prevents cross-chain replay
`nonce`	uint256	Sequential tx count from sender
`maxPriorityFeePerGas`	uint256	Max tip to validator (wei/gas)
`maxFeePerGas`	uint256	Max total fee (wei/gas)
`gasLimit`	uint256	Max gas units allowed
`to`	address	Recipient (`null` = deploy)
`value`	uint256	ETH to transfer (wei)
`data`	bytes	Calldata / initcode
`accessList`	list	Pre-warmed slots
`v, r, s`		ECDSA signature

9.3 Signing and Sender Recovery

No explicit `from` field; the sender is recovered from the signature:

$$\texttt{sigHash} = \text{keccak256}(\text{RLP}(2,\ \texttt{chainId},\ \texttt{nonce}, \ldots, \texttt{accessList})) \tag{9.1}$$

$$(v, r, s) = \text{ECDSA_sign}(\texttt{sigHash},\ \text{private key}) \tag{9.2}$$

$$\texttt{sender} = \text{ecrecover}(v, r, s,\ \texttt{sigHash}) \tag{9.3}$$

9.4 Transaction Execution Flow

1. Validate: `nonce` matches, sender balance $\geq$ `gasLimit`$\times$`maxFeePerGas`.
2. Deduct `gasLimit`$\times$`maxFeePerGas` from sender upfront.
3. Execute EVM code (or simple value transfer).
4. Refund unused gas: (`gasLimit` $-$ `gasUsed`) $\times$ `effectivePrice`.
5. **Burn** base fee: `gasUsed` $\times$ `baseFee`.
6. Pay validator: `gasUsed` $\times$ `priorityFee`.

9.5 Contract Creation

When `to` is null:

- `data` = **initcode** (constructor bytecode + arguments).
- EVM runs initcode; constructor returns runtime bytecode.
- Contract address (CREATE): $\texttt{addr} = \text{keccak256}(\text{RLP}(\texttt{sender}, \texttt{nonce}))[12{:}]$
- Contract address (CREATE2): $\texttt{addr} = \text{keccak256}(\texttt{0xff} \| \texttt{sender} \| \texttt{salt} \| \text{keccak256}(\text{initcode}))[12{:}]$

9.6 Function Calls

`data` = 4-byte selector $\|$ ABI-encoded arguments.

$$\texttt{selector} = \text{keccak256}(\text{"funcName(type1,type2,...)"})[0:4] \tag{9.4}$$

Example — calling `transfer(address,uint256)`:

```
selector  = 0xa9059cbb
calldata  = selector || abi.encode(to, amount)
```

9.7 Internal Calls

Table 9.2: EVM call types

Opcode	**Context**	**State changes**
`CALL`	Callee's context	Yes
`STATICCALL`	Callee's context	No (read-only)
`DELEGATECALL`	Caller's context	Yes (in caller's storage)

Chapter 10

Ethereum Virtual Machine (EVM)

10.1 Architecture

The EVM is a stack-based, 256-bit quasi-Turing-complete virtual machine:

- **Stack**: LIFO, max 1024 elements, each 256 bits.
- **Memory**: Volatile byte array, expands on demand (cost $\propto$ size^2 for large allocations).
- **Storage**: Persistent 256-bit $\rightarrow$ 256-bit key-value map, per contract.
- **Calldata**: Read-only input from the calling transaction.
- **Code**: Read-only bytecode being executed.

10.2 Instruction Set

10.3 Execution Trace Example

```
PUSH1 0x03   // Stack: [3]
PUSH1 0x05   // Stack: [5, 3]
ADD          // Stack: [8]
```

Listing 10.1: Computing $3 + 5$ in EVM bytecode

10.4 Gas Accounting in Practice

Example 10.1. Executing a simple ERC-20 transfer:

Approximate gas breakdown for `transfer(address,uint256)`:

Table 10.1: Key EVM opcodes and gas costs

Category	**Opcode**	**Gas**	**Effect**
Arithmetic	`ADD`	3	$a + b$
	`MUL`	5	$a \times b$
	`EXP`	10+	a^b
Bitwise	`AND`, `OR`, `XOR`	3	bitwise ops
	`SHL`, `SHR`	3	shifts
Memory	`MLOAD`	3+	load 32 B
	`MSTORE`	3+	store 32 B
Storage	`SLOAD`	2100	load slot
	`SSTORE`	20 000	write new slot
Control	`JUMP`, `JUMPI`	8/10	unconditional/conditional
	`REVERT`	0	revert + return data
	`RETURN`	0	return data
Context	`CALLER`	2	`msg.sender`
	`CALLVALUE`	2	`msg.value`
	`TIMESTAMP`	2	block timestamp
Hash	`KECCAK256`	30+6/word	keccak256(mem range)
Calls	`CALL`	100+	external call
	`STATICCALL`	100+	read-only call
	`DELEGATECALL`	100+	delegate call

10.4.1 Out-of-Gas and Reverts

If gas is exhausted mid-execution:

- An **out-of-gas** exception is raised.
- All state changes from this call (and any sub-calls) are *reverted*.
- All remaining gas is consumed (none refunded).
- The transaction is included in the block (fee paid) but has status 0 (failed).

If `REVERT` is executed explicitly:

- State changes are reverted.
- *Unused* gas is refunded.
- A return data (error message) is available.

10.4.2 Call Stack and Call Depth Limit

When a contract calls another, a new **call frame** is pushed onto the call stack:

Operation	**Gas**
Base transaction fee	21 000
Non-zero calldata (68 bytes × 16)	1 088
SLOAD (read sender balance)	2 100
SLOAD (read recipient balance)	2 100
SSTORE (update sender balance)	5 000
SSTORE (update recipient balance)	20 000
LOG3 (Transfer event)	1 500
Misc arithmetic/logic	≈ 500
Total	≈ 53 288

- Each frame has its own **stack**, **memory**, and **program counter**.
- Storage and balance are global (shared across frames).
- The call depth limit is **1024**. Exceeding it causes an out-of-gas-like failure.
- The EIP-150 **63/64 rule**: a CALL forwards at most $\frac{63}{64}$ of remaining gas, ensuring gas eventually runs out even in deep recursion.

10.5 ABI Encoding

Static types (`uint256`, `address`, `bool`): padded to 32 bytes, left-aligned.

Dynamic types (`bytes`, `string`, dynamic arrays): encoded with a head containing an offset to the tail data.

10.6 Contract Deployment Bytecode

1. **Constructor code**: copies runtime bytecode to memory, returns it.
2. **Runtime bytecode**: stored at contract address; executed on calls.
3. **Constructor arguments**: ABI-encoded, appended after runtime bytecode.

Exam Note

`SSTORE` to a new (zero) slot costs 20 000 gas; updating an existing slot costs 5 000; clearing to zero gives a 15 000 gas refund (capped at 20% of gas used in EIP-3529).

Chapter 11

Ethereum Blocks

11.1 Block Structure

An Ethereum block = **execution payload** (transactions, state) wrapped in a **beacon block** (consensus metadata).

11.1.1 Execution Block Header

Table 11.1: Execution layer block header (post-merge)

Field	Type	Description
`parentHash`	bytes32	Hash of parent block header
`ommersHash`	bytes32	keccak256 of uncle list (always empty post-merge)
`beneficiary`	address	Fee recipient
`stateRoot`	bytes32	Root of state MPT after all txs
`transactionsRoot`	bytes32	Root of tx trie
`receiptsRoot`	bytes32	Root of receipts trie
`logsBloom`	bytes256	Bloom filter for logs
`difficulty`	uint256	0 post-merge
`number`	uint256	Block height
`gasLimit`	uint256	Max total gas
`gasUsed`	uint256	Total gas consumed
`timestamp`	uint64	Unix time (exact 12 s slots)
`extraData`	bytes	Up to 32 bytes arbitrary
`prevRandao`	bytes32	Beacon chain RANDAO mix
`nonce`	bytes8	0 post-merge
`baseFeePerGas`	uint256	EIP-1559 base fee
`withdrawalsRoot`	bytes32	Validator withdrawals (EIP-4895)

11.2 Transaction Receipts

- `status`: 1 = success, 0 = revert.

- `cumulativeGasUsed`: running total in block.
- `logs`: events emitted.
- `logsBloom`: Bloom contribution.

11.3 Logs and Events

```
event Transfer(address indexed from, address indexed to, uint256
    value);
```

Log structure:

- Topic 0: `keccak256("Transfer(address,address,uint256)")`
- Topic 1: `from` (indexed)
- Topic 2: `to` (indexed)
- Data: ABI-encoded `value`

Events are *not* accessible from within the EVM; they are only visible to external observers.

11.4 The Merge (September 15, 2022)

- Energy reduced 99.95%: 5.13 GW → 2.62 MW.
- PoW nonce/difficulty set to 0.
- `prevRandao` replaces `mixHash`, injecting beacon chain randomness.
- Block time became exactly 12 s (one slot).

Chapter 12

Solidity Programming

12.1 Language Overview

Solidity is a statically-typed, object-oriented language that compiles to EVM bytecode. It supports contracts, inheritance, events, and custom modifiers.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {
    uint256 private value;

    function set(uint256 _v) public { value = _v; }

    function get() public view returns (uint256) { return value;
        }
}
```

Listing 12.1: Minimal contract

12.2 Types

12.2.1 Value Types

`uint8`–`uint256`, `int8`–`int256`, `bool`, `address` / `address payable`, `bytes1`–`bytes32`, `enum`.

12.2.2 Reference Types

`bytes` (dynamic), `string`, arrays (`T[]`), `struct`, `mapping(K => V)`.

12.2.3 Special Variables

- `msg.sender` — caller address.

- `msg.value` — ETH sent (wei).
- `block.timestamp`, `block.number`.
- `tx.origin` — original EOA (avoid for auth).

12.3 Function Visibility and Mutability

Table 12.1: Function visibility modifiers

Visibility	Accessible from
`public`	Anywhere (EOAs, other contracts, internally)
`external`	Only from outside (more efficient calldata access)
`internal`	Within contract and derived contracts
`private`	Only within this contract

State mutability: `view` (reads state), `pure` (no state access), `payable` (can receive ETH), default (reads + writes).

12.4 Mappings, Structs, Events

```
contract Bank {
    mapping(address => uint256) public balances;
    event Deposit(address indexed user, uint256 amount);
    event Withdrawal(address indexed user, uint256 amount);

    function deposit() public payable {
        balances[msg.sender] += msg.value;
        emit Deposit(msg.sender, msg.value);
    }

    function withdraw(uint256 amount) public {
        require(balances[msg.sender] >= amount, "Insufficient");
        balances[msg.sender] -= amount;        // effect first!
        (bool ok, ) = msg.sender.call{value: amount}("");
        require(ok, "Transfer failed");
        emit Withdrawal(msg.sender, amount);
    }
}
```

Listing 12.2: Bank contract with events

Reentrancy

Always update state *before* external calls. The DAO hack (2016) drained $60 M by re-entering a withdraw function before the balance was updated. Pattern: **Checks** → **Effects** → **Interactions**.

12.5 Modifiers and Inheritance

```
contract Ownable {
    address public owner;
    constructor() { owner = msg.sender; }

    modifier onlyOwner() {
        require(msg.sender == owner, "Not owner");
        _;
    }
}

contract Token is Ownable {
    mapping(address => uint256) public balanceOf;

    function mint(address to, uint256 amount) public onlyOwner {
        balanceOf[to] += amount;
    }
}
```

Listing 12.3: Ownable pattern

12.6 Minimal ERC-20 Implementation

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract ERC20 {
    string  public name;
    string  public symbol;
    uint8   public decimals = 18;
    uint256 public totalSupply;

    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public
       allowance;

    event Transfer(address indexed from, address indexed to,
       uint256 value);
    event Approval(address indexed owner, address indexed
       spender, uint256 value);

    constructor(string memory _name, string memory _symbol,
       uint256 _supply) {
        name = _name; symbol = _symbol;
        totalSupply = _supply * 10**decimals;
        balanceOf[msg.sender] = totalSupply;
    }

```

```
    function transfer(address to, uint256 v) public returns
        (bool) {
        require(balanceOf[msg.sender] >= v);
        balanceOf[msg.sender] -= v;
        balanceOf[to] += v;
        emit Transfer(msg.sender, to, v);
        return true;
    }

    function approve(address spender, uint256 v) public returns
        (bool) {
        allowance[msg.sender][spender] = v;
        emit Approval(msg.sender, spender, v);
        return true;
    }

    function transferFrom(address from, address to, uint256 v)
        public returns (bool)
    {
        require(balanceOf[from] >= v &&
            allowance[from][msg.sender] >= v);
        allowance[from][msg.sender] -= v;
        balanceOf[from] -= v;
        balanceOf[to]   += v;
        emit Transfer(from, to, v);
        return true;
    }
}
```

Listing 12.4: ERC-20 token

12.7 Storage Layout

State variables are stored in 32-byte **slots** starting at slot 0:

- Variables are packed into the same slot if they fit.
- Mappings: `mapping(K=>V)` stores value at `keccak256(K || slot)`.
- Dynamic arrays: length at slot s; elements at `keccak256(s) + i`.

```
contract Layout {
    uint128 a;     // slot 0, bytes 0-15
    uint128 b;     // slot 0, bytes 16-31  (packed with a)
    uint256 c;     // slot 1               (full slot)
    address d;     // slot 2, bytes 0-19
    bool    e;     // slot 2, byte 20      (packed with d)
    mapping(address => uint256) balances; // slot 3 (base)
    // balances[addr] is at keccak256(abi.encode(addr, 3))
}
```

Listing 12.5: Storage layout example

Understanding storage layout is critical for:

- Writing `delegatecall`-based proxies (storage must match).
- Optimising gas (packing saves SSTORE costs).
- Security auditing (spotting storage collisions).

12.8 Interfaces and Abstract Contracts

```
interface IERC20 {
    function totalSupply() external view returns (uint256);
    function balanceOf(address) external view returns (uint256);
    function transfer(address to, uint256 amount) external
       returns (bool);
    function approve(address spender, uint256 amount) external
       returns (bool);
    function transferFrom(address from, address to, uint256
       amount)
        external returns (bool);
    event Transfer(address indexed from, address indexed to,
       uint256 value);
    event Approval(address indexed owner, address indexed
       spender, uint256 value);
}
```

Listing 12.6: Interface definition

Interfaces allow contracts to interact with any ERC-20 token without knowing its implementation:

```
IERC20 token = IERC20(tokenAddress);
uint256 bal = token.balanceOf(msg.sender);
```

12.9 The Proxy Pattern (Upgradeable Contracts)

Smart contracts are immutable. The proxy pattern separates logic from storage to allow upgrades:

1. A **proxy contract** holds all storage and ETH.
2. It `delegatecall`s a **logic contract** for all function calls.
3. To upgrade: deploy a new logic contract and update the proxy's stored implementation address.

```
contract Proxy {
    address public implementation;   // slot 0
    address public admin;            // slot 1

    constructor(address _impl) {
        implementation = _impl;
        admin = msg.sender;
    }

    function upgrade(address newImpl) external {
        require(msg.sender == admin);
        implementation = newImpl;
    }

    fallback() external payable {
        address impl = implementation;
        assembly {
            calldatacopy(0, 0, calldatasize())
            let result := delegatecall(gas(), impl, 0,
               calldatasize(), 0, 0)
            returndatacopy(0, 0, returndatasize())
            switch result
            case 0 { revert(0, returndatasize()) }
            default { return(0, returndatasize()) }
        }
    }
}
```

Listing 12.7: Minimal transparent proxy

Exam Note

Storage collision hazard: If the proxy stores `implementation` at slot 0 and the logic contract also uses slot 0 for a state variable, a call will overwrite the implementation pointer! EIP-1967 standardises proxy storage slots to `keccak256("eip1967.proxy.implementation") - 1` to avoid this.

12.10 Common Vulnerabilities

Table 12.2: Common Solidity vulnerabilities

Vulnerability	**Description**	**Mitigation**
Reentrancy	Re-enter before state updated	CEI pattern; `nonReentrant`
Integer overflow	Wrap-around (pre-0.8)	Solidity $\geq$ 0.8; SafeMath
Missing access ctrl	No `onlyOwner`	OpenZeppelin AccessControl
Front-running	Mempool visibility	Commit-reveal
`tx.origin` auth	Phishing via forwarding	Always use `msg.sender`
`delegatecall`	Storage layout clash	Careful proxy patterns

Chapter 13

JavaScript and Node.js

13.1 JavaScript Essentials

```
let x = 1;            // block-scoped, reassignable
const PI = 3.14;      // block-scoped, constant
var y = 2;            // function-scoped -- avoid

// Arrow function
const add = (a, b) => a + b;

// Arrow with block body
const safeDivide = (a, b) => {
    if (b === 0) throw new Error("Division by zero");
    return a / b;
};
```

Listing 13.1: Variables and functions

13.2 Arrays

```
let arr = [1, 2, 'Hello'];
arr.push(4);            // Add to end
arr.pop();              // Remove from end
arr.unshift('A');       // Add to start
arr.shift();            // Remove from start

// Functional methods
arr.map(x => x * 2);
arr.filter(x => typeof x === 'number');
arr.reduce((acc, x) => acc + x, 0);
```

Listing 13.2: Array operations

13.3 Objects

```
let user = { name: "John", age: 30 };

console.log(user.name);       // dot notation
console.log(user["age"]);     // bracket notation

user.isAdmin = true;          // add property
delete user.age;              // remove property
console.log("age" in user);   // false

for (let k in user) { console.log(k, user[k]); }
```

Listing 13.3: Objects

13.4 Promises and Async/Await

Many browser and Node.js APIs are asynchronous.

```
fetch("https://example.com/")
    .then(response => response.text())
    .then(text => console.log(text))
    .catch(e => console.log("Error:", e));
```

Listing 13.4: Promise chain

`async/await` is syntactic sugar making async code look synchronous:

```
async function getText(url) {
    const response = await fetch(url);
    const text     = await response.text();
    console.log(text);
}
getText("https://example.com/");
```

Listing 13.5: async/await

An `async` function always returns a `Promise`. `await` can only be used inside `async` functions.

13.5 Node.js

Node.js enables JavaScript outside the browser:

- Built on Chrome's V8 engine.
- Event-driven, non-blocking I/O.
- **npm** (Node Package Manager): millions of packages.

```
npm init                        # create package.json
npm install ethers              # install locally
npm install -g truffle          # install globally
node script.js                  # run a script
```

Listing 13.6: npm essentials

13.6 Accepting CLI Input with Inquirer

```
import inquirer from "inquirer";   // npm install inquirer

const questions = [
    { name: "name",     type: "input",
      message: "What's your name?" },
    { name: "program", type: "select",
      choices: ["BTech","Dual Degree","MTech","IDDP","PhD"],
      message: "Which program are you in?" },
];

inquirer.prompt(questions).then(answers => {
    console.log(`Name: ${answers.name}`);
    console.log(`Program: ${answers.program}`);
});
```

Listing 13.7: inquirer example (main.js)

`package.json` must include `"type":"module"` to use `import` syntax.

13.7 Connecting to Ethereum with ethers.js

```
import { ethers } from "ethers";

const provider = new ethers.BrowserProvider(window.ethereum);
const signer   = await provider.getSigner();

const contract = new ethers.Contract(
    contractAddress, contractABI, signer);

const balance = await contract.balanceOf(signer.address);

const tx = await contract.transfer(recipient, amount);
await tx.wait();   // wait for 1 confirmation
```

Listing 13.8: ethers.js basics

Chapter 14

Tornado Cash and Privacy

14.1 Motivation

All Ethereum transactions are public. Any observer who knows your address can track every deposit and withdrawal, revealing your total holdings.

Concrete risk: Pay a hotel 1 ETH; the front-desk clerk sees your wallet holds 99 ETH and has kidnapping connections.

Option A — use an exchange: custody risk, KYC leaks.
Option B — send to a fresh address first: repeated transfers still link old and new address.
Tornado Cash is a better Option B: a smart-contract *mixer* that breaks the on-chain link.

14.2 Protocol Overview

1. Alice, Bob, Carol each deposit 1 ETH into the Tornado Cash contract.
2. Later, anyone who holds a valid private note can withdraw 1 ETH to any fresh address.
3. On-chain, the withdrawal cannot be linked to any specific deposit.

14.2.1 Desired Properties

Soundness Only a genuine depositor can withdraw; no double withdrawal.

Privacy A withdrawal is unlinkable to its deposit.

14.3 Commitment Scheme

14.3.1 Private Note

The user's browser generates 62 random bytes:

```
tornado-eth-0.1-5-0x<Header><31-byte Nullifier><31-byte Secret>
```

14.3.2 Pedersen Hash

A **commitment** is the Pedersen hash of the concatenated bytes:

$$C = \text{PedersenHash}(b_1 b_2 \cdots b_n) = g_1^{b_1} g_2^{b_2} \cdots g_n^{b_n} \tag{14.1}$$

where g_i are fixed public generators of an elliptic curve group. The commitment is *hiding* (reveals nothing about nullifier/secret) and *binding* (cannot find two inputs with the same hash).

14.4 Deposit Steps

1. Generate random nullifier and secret on client.
2. Compute $C = \text{PedersenHash}(\text{nullifier}\|\text{secret})$.
3. Send $C + 1$ ETH to contract. Contract checks C is fresh:

```
mapping(bytes32 => bool) public commitments;
require(!commitments[_commitment], "Already submitted");
```

4. Contract inserts C into a 20-level Merkle tree ($\leq 2^{20}$ deposits, no deletions):

```
uint32 idx = _insert(_commitment);
```

5. Contract marks C seen and verifies `msg.value == 1 ETH`.
6. Contract emits `Deposit(_commitment, idx, block.timestamp)`.

14.5 Zero-Knowledge Proofs: zk-SNARKs

14.5.1 What Is a zk-SNARK?

A **zk-SNARK** (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) lets a prover convince a verifier of a statement's truth without revealing any secret inputs.

14.5.2 The Withdrawal Statement

> *I know a* nullifier *and* secret *such that:*
> *(1)* PedersenHash(*nullifier*‖*secret*) *is a leaf of the Merkle tree with root* `_root`*, and*
> *(2)* PedersenHash(*nullifier*) = `_nullifierHash`*.*

The prover generates a proof of this statement and submits it on-chain without revealing which commitment (leaf) it corresponds to.

Table 14.1: zk-SNARK properties

Property	Meaning
Completeness	Honest prover always generates valid proof
Soundness	Infeasible to prove a false statement
Zero-Knowledge	Proof reveals nothing beyond statement truth
Succinct	Proof is short and fast to verify
Non-Interactive	No back-and-forth; single proof string

14.6 Withdrawal Steps

```
function withdraw(
    bytes  calldata _proof,
    bytes32 _root,
    bytes32 _nullifierHash,
    address payable _recipient,
    address payable _relayer,
    uint256 _fee
) external;
```

Listing 14.1: withdraw() signature

1. Check nullifier not yet spent (prevents double withdrawal):

```
mapping(bytes32 => bool) public nullifierHashes;
require(!nullifierHashes[_nullifierHash], "Note already
    spent");
```

2. Check `_root` is one of the last 100 known roots (allows deposits while withdrawals are pending):

```
require(isKnownRoot(_root), "Cannot find your merkle root");
```

3. Verify SNARK proof on-chain:

```
require(verifier.verifyProof(_proof,
    [uint256(_root), uint256(_nullifierHash), ...]),
    "Invalid withdraw proof");
```

4. Mark nullifier spent; send ETH to recipient minus relayer fee.

Relayer: Pays gas for the user (whose fresh address has no ETH) and receives a fee deducted from the denomination.

14.7 Groth16: The Proof System Used by Tornado Cash

Tornado Cash uses the **Groth16** zk-SNARK proof system (Jens Groth, 2016).

14.7.1 Arithmetic Circuits

Any computation can be represented as an **arithmetic circuit** over a finite field $\mathbb{F}_p$: wires carry field elements, and gates compute addition (+) and multiplication (×) in $\mathbb{F}_p$.

A **Rank-1 Constraint System (R1CS)** is a set of constraints of the form:

$$(\vec{a} \cdot \vec{w}) \times (\vec{b} \cdot \vec{w}) = (\vec{c} \cdot \vec{w})$$

where $\vec{w}$ is the witness vector (all wire values: public inputs, private inputs, and intermediate values).

14.7.2 Quadratic Arithmetic Programs (QAP)

Groth16 converts R1CS into a **QAP**: polynomials $A(x), B(x), C(x)$ such that the constraint is satisfied iff $A(x) \cdot B(x) - C(x) = H(x) \cdot Z(x)$ where $Z(x) = \prod_i (x - r_i)$ is the vanishing polynomial at the constraint evaluation points.

14.7.3 Groth16 Proof and Verification

Trusted Setup: A one-time ceremony generates **Structured Reference Strings (SRS)** (σ_1, σ_2) containing encrypted powers of a secret τ. The secret τ must be destroyed after setup ("toxic waste").

Proof: Three elliptic curve points $\pi = (A, B, C) \in G_1 \times G_2 \times G_1$. Constant size ($\approx$ 200 bytes) regardless of circuit complexity.

Verification: Check two pairing equations:

$$e(A, B) = e(\alpha, \beta) \cdot e(\sum_i a_i u_i, \gamma) \cdot e(C, \delta)$$

Verification takes $\approx$ 2 ms on modern hardware (only 2 pairings).

14.7.4 Poseidon Hash

The original Tornado Cash uses Pedersen hash. More modern ZK systems use **Poseidon**, a hash function designed specifically for arithmetic circuits:

- Based on a sponge construction with a round function over $\mathbb{F}_p$.
- Far fewer constraints than SHA-256 ($\approx$ 240 vs $\approx$ 27 000 constraints per 256-bit hash).
- The round function: $f(x) = (x + c)^5 \bmod p$ (simple multiplication gate).

14.7.5 Trusted Setup Ceremonies

The Groth16 setup is **circuit-specific**: a new ceremony is needed for each circuit. Tornado Cash used a multi-party computation (MPC) ceremony with 1114 participants. The system is secure as long as *at least one participant* honestly destroys their secret.

For universal setups (Plonk, STARKs), a single ceremony works for all circuits up to a given size.

14.8 The Circom Circuit

The circuit is compiled with **circom** and a verifier contract is generated automatically.

```
template Withdraw(levels) {
    signal input  root;
    signal input  nullifierHash;
    signal input  recipient;  // not constrained -- prevents
       tampering
    signal input  relayer;
    signal input  fee;
    signal private input nullifier;
    signal private input secret;
    signal private input pathElements[levels];
    signal private input pathIndices[levels];

    // 1. Verify nullifierHash = PedersenHash(nullifier)
    component hasher = CommitmentHasher();
    hasher.nullifier <== nullifier;
    hasher.secret    <== secret;
    hasher.nullifierHash === nullifierHash;

    // 2. Verify commitment is in Merkle tree with root `root`
    component tree = MerkleTreeChecker(levels);
    tree.leaf <== hasher.commitment;
    tree.root <== root;
    for (var i = 0; i < levels; i++) {
        tree.pathElements[i] <== pathElements[i];
        tree.pathIndices[i]  <== pathIndices[i];
    }

    // Prevent optimiser from removing recipient/relayer/fee
    signal rSq; rSq <== recipient * recipient;
    signal fSq; fSq <== fee      * fee;
    signal lSq; lSq <== relayer  * relayer;
}
component main = Withdraw(20);
```

Listing 14.2: withdraw.circom (simplified)

14.9 OFAC Sanctions

- **Aug 8, 2022**: OFAC added Tornado Cash contracts to its sanctions list, alleging facilitation of ransomware money laundering.
- GitHub removed repos; three contributors suspended.

- Developer **Alexey Pertsev** arrested (Netherlands, Aug 2022); sentenced 64 months (May 2024); released Feb 2025.
- Developer **Roman Storm** arrested (US, Aug 2023); case ongoing.
- EFF and Prof. Matthew Green's advocacy led OFAC to allow educational use of the code.
- GitHub repos restored in 2023.

Exam Note

Tornado Cash illustrates the collision between financial privacy (a legitimate right) and AML regulations. Core question: can *open-source code* — not its operators — be sanctioned?

Chapter 15

Consensus Protocols

15.1 State Machine Replication (SMR)

Definition

State Machine Replication: n nodes each maintain a local append-only *history* (ordered list of transactions).

- **Consistency (Safety)**: All nodes agree on the same history.
- **Liveness**: Every valid client transaction is eventually added to the history.

If all nodes start from the same initial state and apply the same transactions in the same order, they reach identical final states.

15.2 Network Models

Table 15.1: Network models for distributed consensus

Model	Clock	Message delay
Synchronous	Global, shared	Bounded: msg at step t arrives by $t+1$
Asynchronous	None	Unbounded (messages eventually arrive)
Partially synchronous	Global, shared	Unbounded before GST; $\leq \Delta$ after GST

GST (Global Stabilisation Time): exists but is unknown in advance.

15.3 Fault Types

Let f = Byzantine nodes, n = total.

Table 15.2: Node fault types

Fault	Behaviour	Severity
Crash	Stops working at time t	Low
Omission	Drops some messages	Medium
Byzantine	Deviates arbitrarily; cannot forge signatures	Highest

15.4 SMR with Rotating Leaders (Honest Setting)

Assumptions: Permissioned (n known), PKI, synchronous, $f = 0$.

- In time step t: leader $= 1 + (t \bmod n)$.
- Leader broadcasts ordered list of new transactions.
- Every node appends the list.

Consistency: All nodes receive same broadcast $\Rightarrow$ identical histories.

Liveness: Transaction submitted to node i is included when node i becomes leader.

15.5 Byzantine Broadcast (BB)

Definition

One of n nodes is the *sender* (may be Byzantine) with private input $v^* \in V$:

Termination Every honest node eventually halts with some $v_i \in V$.

Agreement All honest nodes output the same value.

Validity If sender is honest, all output v^*.

SMR reduces to BB via rotating leaders: each time step invokes BB with the current leader as sender.

15.5.1 Protocol for $f = 1$ (Cross-Checking)

Three rounds:

1. Sender broadcasts v^* with signature.
2. Each non-sender signs received message m_i, broadcasts (m_i, σ_i). Uses $\perp$ if nothing received.
3. Each non-sender takes majority of received messages.

Proposition: Satisfies termination, agreement, validity for $f = 1$.

Fails for $f = 2$: Byzantine sender splits votes; Byzantine non-sender amplifies the split; two groups of honest nodes disagree.

15.6 The Dolev-Strong Protocol

Theorem 15.1 (Dolev-Strong, 1983)**.** *In the permissioned, PKI, synchronous setting, there exists a protocol for BB running in $f+1$ rounds that satisfies termination, agreement, and validity for any f.*

Convinced message: node i is *convinced of v at step t* if it receives a message referencing v signed by the sender and $\geq t-1$ other distinct nodes.

Protocol:

- Step 0: Sender sends v^* + signature; outputs v^*.
- Steps $t = 1, \ldots, f+1$: If convinced of v and not previously, sign and broadcast.
- Final: If convinced of exactly one value v, output v; else $\perp$.

15.7 Correctness of Dolev-Strong

Proof of Agreement. Suppose honest nodes i and j both output values at the end. We show they output the same value.

For a value v to be output by honest node i, node i must have been convinced of v at some time step $t \leq f+1$. Being convinced means i received a message for v signed by the sender and by $t-1$ other distinct nodes.

Claim: If honest node i is first convinced of v at step t, then all honest nodes are convinced of v by step $t+1$.

Reason: At step t, node i signs v and broadcasts. Every honest node then receives this message signed by i plus the $t-1$ other signatures (total: t signatures including sender). So every honest node is convinced of v by step $t+1 \leq f+2$, before the protocol ends at step $f+1$.

Wait — the protocol ends at step $f+1$ and conviction spreads by step $f+2$? We need $t \leq f$. If i is first convinced at step $f+1$ (the final step), it still broadcasts, but other nodes only check at the end of step $f+1$. Since all messages arrive by step $f+2$ in the synchronous model, and all honest nodes output at the end of step $f+1$, we need to be more careful.

The correct argument: honest node i is convinced of v at step $t \leq f+1$. At step t, i re-signs and broadcasts. Any honest non-sender j will receive, by step $t+1$, a message for v signed by the sender and t other nodes (including i). So j is convinced of v by step $t+1 \leq f+2$. But wait, the protocol runs for $f+2$ steps total (steps $0, 1, \ldots, f+1$), so at step $f+1$ (last step), j receives the forwarded signatures and becomes convinced. Both i and j are convinced of v, and since this process is symmetric, the only value any honest node can be convinced of is the one that spreads this way.

If a Byzantine sender sends different values to different nodes, at most one chain of signatures can grow to convince honest nodes, because digital signatures cannot be forged. □ □

Proof of Validity. If the sender is honest, it sends v^* with its signature in step 0. Every honest non-sender receives a message signed by the sender for v^* by step 1, so they are

convinced of v^*. They re-broadcast and all honest nodes output v^*. No other value has a sender signature (honest sender sends only v^*), so no honest node is convinced of anything else. □ □

15.8 Impossibility Results

15.8.1 FLM (No-PKI Bound)

Theorem 15.2 (Pease–Shostak–Lamport 1980; Fischer–Lynch–Merrit 1986)**.**

$$\boxed{f \geq \frac{n}{3} \;\Rightarrow\; \textit{no BB protocol without PKI}}$$

PKI is *necessary* for Dolev-Strong.

15.8.2 FLP (Asynchronous Impossibility)

Theorem 15.3 (Fischer–Lynch–Paterson, 1985)**.** *For $n \geq 2$, even with $f = 1$, no* deterministic *protocol solves Byzantine Agreement in the asynchronous model.*

Practical blockchains escape FLP by using the **partially synchronous** model.

15.8.3 Partially Synchronous Bound

Theorem 15.4. *There exists a deterministic BFT protocol for partially synchronous networks iff $f < \frac{n}{3}$. Safety holds at all times; liveness holds post-GST.*

Tendermint and PBFT achieve this bound.

15.9 Consensus Summary Table

Table 15.3: Consensus protocol comparison

Protocol	**Network model**	**Max f**	**Guarantees**	**Used by**
Rotating leaders	Sync, honest	0	C + L	Toy
Dolev-Strong	Sync, PKI	Any	T + A + V	Academic
PBFT	Partial sync	$< n/3$	C + L*	HyperLedger
Tendermint	Partial sync	$< n/3$	C + L*	Cosmos
Nakamoto	Eventual sync	$<$ 50% hash	Prob. C + L	Bitcoin
Casper FFG	Partial sync	$< n/3$ stake	C + L* + slash	Ethereum

*Liveness only post-GST.

Chapter 16

Tendermint Protocol

16.1 Overview

Tendermint is a BFT consensus protocol achieving:

- Consistency and eventual (post-GST) liveness.
- Optimal fault tolerance: $f < n/3$.
- Partially synchronous network, permissioned, PKI.

Key ideas: rotating leaders, two stages of voting per round, restarts after timeout (if messages are delayed), weighted voting in practice (proportional to stake).

16.2 Rounds and Phases

A **round** spans 4Δ time steps:

Table 16.1: Tendermint round phases

Phase	**Start time**	**Action**
1 (Propose)	$4\Delta r$	Leader broadcasts (B_i, Q_i)
2 (Prevote)	$4\Delta r + \Delta$	Nodes cast stage-1 votes
3 (Precommit)	$4\Delta r + 2\Delta$	Nodes cast stage-2 votes
4 (Commit)	$4\Delta r + 3\Delta$	Commit if $\geq \frac{2}{3}n$ stage-2 votes

Each round has a unique **leader**; if no consensus, nodes move to round $r + 1$.

16.3 Quorum Certificates

Each vote has attributes: (voter i, block B, height h, round r, stage s). The triple (h, r, s) is a **referendum**.

Definition

A **Quorum Certificate (QC)** for a referendum (h, r, s) is a set of $\geq \frac{2}{3}n$ votes all for the same block in that referendum.

Lemma 16.1. *Any two QCs share* $\geq \frac{n}{3}$ *voters.*

Corollary 16.2. *If* $f < \frac{n}{3}$*, any two QCs for the same referendum support the* same *block (since they share at least one honest voter who votes at most once).*

16.4 Local State per Node

Node i working on block height h_i maintains:

- B_i: current belief about the correct block at height h_i.
- Q_i: most recent QC supporting B_i (null initially).

QC ordering: $(h, r_1, s_1) >_{age} (h, r_2, s_2)$ iff $r_1 > r_2$, or $r_1 = r_2$ and $s_1 > s_2$.

16.5 Protocol Pseudocode

```
if i == l then                    // i is the leader
    if l received a height-h QC newer than (B_i, Q_i) then
        B_i := B_l;  Q_i := Q_l
    end
    broadcast(B_i, Q_i)           // annotated with h_i, r,
       signature
end
```

Listing 16.1: Phase 1: Leader proposes

```
if i received (B_l, Q_l) from leader l then
    if Q_l is at least as recent as Q_i then
        B_i := B_l;  Q_i := Q_l
        broadcast(B_i, Q_i)
        broadcast stage-1-vote for B_i   // annotated h_i, r, sig
    end
end
```

Listing 16.2: Phase 2: Stage-1 vote

```
if i received >= (2n/3) round-r stage-1 votes for B then
    B_i := B;  Q_i := (those votes)      // round-r stage-1 QC
    broadcast(B_i, Q_i)
    broadcast stage-2-vote for B_i
end
```

Listing 16.3: Phase 3: Stage-2 vote

```
if i received >= (2n/3) round-r stage-2 votes for B then
    B_i := B;  Q_i := (those votes)      // round-r stage-2 QC
    broadcast(B_i, Q_i)
    commit B_i to local history at height h_i
    h_i := h_i + 1
    reset B_i; reset Q_i to null
end
```

Listing 16.4: Phase 4: Commit

Addendum: If stage-2 QCs for height h_i arrive before Phase 1 of round $r+1$, execute Phase 4 immediately.

16.6 Proof of Consistency

Theorem 16.3. *If $f < n/3$ and honest nodes i, j both commit blocks B, B' at the same height h, then $B = B'$.*

Proof sketch:

1. Let r be the first round in which $> \frac{n}{3}$ honest nodes contribute height-h stage-2 votes for a common block B^*. Call this set S.
2. Any different block $B' \neq B^*$ committed in round $r+1$ would need a stage-2 QC in referendum $(h, r+1, 2)$.
3. Stage-2 QC requires $\geq \frac{2}{3}n$ votes; $|S| > \frac{n}{3}$ so at least one node from S must contribute.
4. Nodes in S hold Q_i = stage-1 QC$(h, r, 1)$ supporting B^*. They only cast stage-1 votes for a block whose QC is at least as recent as Q_i. Stage-1 QC$(h, r+1, 1)$ is more recent than stage-1 QC$(h, r, 1)$, but it too must support B^* (same argument applied inductively).
5. Therefore B^* is the only block that can be committed. □

16.7 Liveness in Tendermint (Post-GST)

Theorem 16.4. *After GST, if $f < n/3$, Tendermint terminates (adds a block to history) within $O(\Delta)$ time.*

Argument: After GST, all messages arrive within Δ time. Consider the first round r that starts after GST:

1. The leader l broadcasts (B_l, Q_l) in Phase 1. Since $f < n/3$, at least $\frac{2}{3}n$ honest nodes receive this message by the end of Phase 2.
2. At least $\frac{2}{3}n$ honest nodes cast stage-1 votes for B_l (assuming Q_l is at least as recent as their local Q).

3. Every honest node receives $\frac{2}{3}n$ stage-1 votes by Phase 3, forming a stage-1 QC, and casts a stage-2 vote.

4. Every honest node receives $\frac{2}{3}n$ stage-2 votes by Phase 4 and commits.

The "at least as recent" condition for stage-1 votes is subtle. The consistency proof shows that any block proposed by an honest leader in round $r \geq r^*$ (where r^* is the first commitment round) will be compatible with previously committed values.

16.8 Safety vs. Liveness in Partially Synchronous Protocols

A fundamental tension in consensus:

- **Safety** (consistency): no two honest nodes commit different values. This must hold *at all times*, including before GST.
- **Liveness**: transactions are eventually committed. This can only be guaranteed *after GST*.

Why safety before GST? If the network is partitioned (two groups cannot communicate), each group might commit a different block (split-brain). Tendermint avoids this by requiring $\frac{2}{3}n$ votes for commitment; in a partition where each side has $< \frac{2}{3}n$ nodes, *neither side* can commit. This sacrifices liveness (no progress) but preserves safety (no inconsistency).

This is the essence of the **CAP theorem**: under a network partition, a distributed system must choose between consistency and availability. Tendermint chooses consistency.

16.9 Practical Deployment

- Cosmos SDK, Binance Smart Chain, Polygon.
- Nodes cast **weighted votes** (stake proportional), so quorums require $> \frac{2}{3}$ of total stake.
- See `mintscan.io` for live deployments.
- **Fast finality**: unlike Nakamoto consensus (probabilistic finality), Tendermint achieves *instant* finality once a block is committed — it can never be reverted.

Chapter 17

Ethereum Proof-of-Stake

17.1 The Merge

September 15, 2022: Ethereum transitioned to PoS:

- Energy reduced 99.95%: 5.13 GW $\rightarrow$ 2.62 MW.
- Block time became deterministic: 12 s per slot.
- PoW nonce/difficulty obsoleted (set to 0).

17.2 Node Architecture

Execution client Executes transactions, maintains mempool, updates world state (EVM layer).

Beacon chain client Runs the PoS algorithm; achieves consensus on execution payloads.

17.3 Validators

- Must stake **32 ETH** to the deposit contract.
- $\approx$ 980 000 validators in March 2024.
- Earn attestation and proposal rewards.
- Slashed (penalised + forced exit) for double voting or surround voting.

17.4 Slots and Epochs

- **Slot**: 12 seconds; one block proposed per slot.
- **Epoch**: 32 slots ($\approx$ 6.4 minutes).

- Each slot: $\frac{1}{32}$ of all validators are active.
- One active validator per slot is the **block proposer**.

17.5 Committees and Attestations

Validators in a slot are divided into **committees**:

- $128 \leq$ committee size ≤ 2048; max 64 committees.

Each validator creates an **attestation** (signed vote):

```
class AttestationData:
    slot:                uint64
    committee_index:     uint64
    beacon_block_root:   bytes32    # LMD GHOST vote
    source:              Checkpoint # Casper FFG source
    target:              Checkpoint # Casper FFG target

class Attestation:
    aggregation_bits:    Bitlist[MAX_VALIDATORS_PER_COMMITTEE]
    data:                AttestationData
    signature:           BLSSignature
```

Listing 17.1: Attestation data structure

Aggregators: subset of committee; aggregate attestations using BLS before forwarding to the next block proposer.

17.6 BLS Signatures

BLS (Boneh–Lynn–Shacham) signatures support efficient aggregation: n signatures on the same message aggregate to a single signature of the same size.

17.6.1 Mathematical Foundation

Let G_1, G_2 be elliptic curves of prime order p with generators g_1, g_2. Let G_T be a multiplicative subgroup of a finite field.

Bilinear pairing: $e : G_1 \times G_2 \to G_T$ with

1. $e(g_1^\alpha, g_2^\beta) = e(g_1, g_2)^{\alpha\beta}$ (bilinearity)
2. $e(g_1, g_2) \neq 1$ (non-degeneracy)

Key generation: $sk = x$, $pk = g_1^x$.
Sign: $\sigma = H(m)^x \in G_2$ where $H : \{0,1\}^* \to G_2$.
Verify: $e(pk, H(m)) \stackrel{?}{=} e(g_1, \sigma)$.

17.6.2 Aggregation

For n validators with signatures $\sigma_1, \ldots, \sigma_n$ on the same message:

$$\sigma_{\text{agg}} = \prod_{i=1}^{n} \sigma_i \tag{17.1}$$

$$pk_{\text{agg}} = \prod_{i=1}^{n} pk_i \tag{17.2}$$

Verify: $e(pk_{\text{agg}}, H(m)) \stackrel{?}{=} e(g_1, \sigma_{\text{agg}})$. Constant-size proof of n validators.

17.7 Validator and Committee Selection

17.7.1 RANDAO: On-Chain Randomness

Committee and block proposer assignment requires **unbiasable, unpredictable randomness**. Ethereum uses **RANDAO**:

1. Each validator holds a private RANDAO secret r_i.
2. In every block they propose, validators reveal $\text{hash}^n(r_i)$ for decreasing n (one preimage per proposal).
3. The beacon chain XORs all reveals: $\text{RANDAO} = v_1 \oplus v_2 \oplus \cdots$.
4. At the end of each epoch, the accumulated RANDAO mix is used as the seed for the next epoch's committee assignments.

Manipulation: A validator revealing last can choose whether to reveal or not, gaining one bit of influence on the outcome. This is a known limitation; Ethereum mitigates it with VDF (Verifiable Delay Functions) proposals for future upgrades.

17.7.2 Committee Assignment Algorithm

At the start of each epoch, validators are pseudorandomly shuffled using **swap-or-not** (a Fisher-Yates variant efficient for proofs):

1. Take the list of active validators.
2. For each round of the shuffle: for each index i, compute a pivot $j = \text{hash}(\text{seed}\|i)$, swap i with j if a coin flip (from hash) says to.
3. The shuffled list is divided into slots and committees.

All validators can independently compute any committee assignment given only the RANDAO seed (no central coordinator needed).

17.8 LMD GHOST Fork Choice Rule

LMD GHOST = Latest Message Driven Greedy Heaviest Observed SubTree.

- **Latest**: only the most recent attestation from each validator counts (equivocation resistance).
- **Message Driven**: fork choice driven by attestations, not just block headers.
- **Greedy Heaviest Subtree**: at each fork, choose the branch with the most total stake weight.

Algorithm: starting at genesis, at each fork compute the cumulative stake of validators whose latest attestation supports each branch; choose the heaviest branch.

17.8.1 LMD GHOST Worked Example

Example 17.1. Consider a block tree after a fork:

```
Genesis -- A -- B -- C          (branch 1)
                \-- D -- E      (branch 2)
```

Suppose 80 validators have attested:

- 30 validators' latest attestation: block C (on branch 1)
- 50 validators' latest attestation: block E (on branch 2)

LMD GHOST: at block B's fork:

- Branch 1 (C): 30 votes
- Branch 2 (D/E): 50 votes

LMD GHOST chooses **branch 2** (heavier), so E is the head.

Key insight: LMD GHOST uses the *latest* vote from each validator. This means a validator who voted for C and then equivocates with a vote for E has their earlier vote *replaced*, not *added*. This prevents a simple attack where a validator accumulates weight by voting many times.

17.9 Casper FFG

Casper FFG (Friendly Finality Gadget) provides economic finality on top of LMD GHOST.

17.9.1 Checkpoints

```
class Checkpoint:
    epoch: uint64     # epoch number
    root:  bytes32    # hash of first block in that epoch
```

The first slot of every epoch is a **checkpoint**.

17.9.2 FFG Votes

An FFG vote is a link $s \to t$ where:

- s = highest justified checkpoint the validator knows (*source*)
- t = current epoch checkpoint (*target*)
- s must be an ancestor of t

A **supermajority link** $s \to t$: $\geq \frac{2}{3}$ of validators voted for it.

17.9.3 Justification

If c_1 is justified and $c_1 \to c_2$ is a supermajority link, c_2 becomes **justified**.

17.9.4 Finalization

If justified $c_1 \to c_2$ is a supermajority link and c_2 is the *direct child epoch* of c_1, then c_1 is **finalized**.

Key Insight

Once a checkpoint is finalized, it is irreversible as long as $< \frac{1}{3}$ of the stake is Byzantine. If two conflicting checkpoints are finalized, at least $\frac{1}{3}$ of the stake *can be identified* and slashed (**accountable safety**).

17.10 Casper FFG Slashing Rules

17.10.1 Rule 1 — No Double Vote

A validator must not cast two distinct votes $s_1 \to t_1$ and $s_2 \to t_2$ with $h(t_1) = h(t_2)$ (same target epoch height).

17.10.2 Rule 2 — No Surround Vote

A validator must not cast $s_1 \to t_1$ and $s_2 \to t_2$ such that:

$$h(s_1) < h(s_2) < h(t_2) < h(t_1)$$

(one vote "surrounds" the other).

17.11 Accountable Safety

Theorem 17.1. *If $< \frac{1}{3}$ of validators are Byzantine and two conflicting checkpoints a_m, b_n $(m < n)$ are both finalized, then at least $\frac{1}{3}$ of total stake can be proven to have violated Rule 1 or Rule 2, and hence slashed.*

Proof sketch: There is a chain of supermajority links from genesis to b_n. The finalization of a_m requires a supermajority link $a_m \to a_{m+1}$. The pair (a_m, a_{m+1}) must fall between two consecutive links $b_{j-1} \to b_j$ in the chain to b_n. Any validator who voted for both $a_m \to a_{m+1}$ and a link in the b_n chain violated Rule 1 (same target epoch) or Rule 2 (surround vote). Since two QCs overlap by $> \frac{1}{3}$ stake, enough violators are identifiable. □

17.12 Ethereum PoS Parameter Summary

Table 17.1: Ethereum PoS key parameters

Parameter	**Value**
Slot duration	12 seconds
Epoch	32 slots $\approx$ 6.4 min
Validator deposit	32 ETH
Validators per slot	$\approx n/32$
Committee size	128–2048
Fault tolerance	$f < \frac{1}{3}$ of total stake
Finality latency	$\approx$ 2 epochs ($\approx$ 12.8 min)
Fork choice	LMD GHOST
Finality gadget	Casper FFG
Signature scheme	BLS12-381

Exam Note

Ethereum PoS vs. Tendermint: Both achieve $f < n/3$ BFT in the partially synchronous model. Tendermint uses one-vote-per-node; Ethereum uses stake-weighted BLS aggregation, enabling participation of $\sim 10^6$ validators at manageable bandwidth. LMD GHOST handles fork choice; Casper FFG adds economic finality with slashing.

Appendix A

Mathematical Quick Reference

A.1 Group Theory

Table A.1: Group theory identities

Statement	**Formula**
Fermat's Little Thm	$a^{p-1} \equiv 1 \pmod{p}$ (p prime, $p \nmid a$)
Modular inverse	$a^{-1} \equiv a^{p-2} \pmod{p}$
Euler's Criterion	$a^{(p-1)/2} \equiv \pm 1 \pmod{p}$
Lagrange's Theorem	$\lvert H\rvert \mid \lvert G\rvert$ for $H \leq G$

A.2 Elliptic Curve Operations

Table A.2: ECC quick reference

Operation	**Formula**	**Complexity**
Addition $P+Q$	$\lambda = \frac{y_2-y_1}{x_2-x_1}$; $x_3 = \lambda^2 - x_1 - x_2$; $y_3 = \lambda(x_1 - x_3) - y_1$	$O(1)$
Doubling $2P$	$\lambda = \frac{3x_1^2+a}{2y_1}$; $x_3 = \lambda^2 - 2x_1$; $y_3 = \lambda(x_1 - x_3) - y_1$	$O(1)$
Scalar mult kP	double-and-add	$O(\log k)$
ECDLP	find k given G, kG	$O(\sqrt{n})$

A.3 Hash Function Summary

Table A.3: Hash functions used in Bitcoin and Ethereum

Function	**Output**	**Preimage**	**Collision**
SHA-256	256 bit	2^{256}	2^{128}
RIPEMD-160	160 bit	2^{160}	2^{80}
SHA256d	256 bit	2^{256}	2^{128}
Keccak-256	256 bit	2^{256}	2^{128}

Appendix B

Glossary

ASIC Application-Specific Integrated Circuit; custom mining chip.

BFT Byzantine Fault Tolerant: works with $< \frac{1}{3}$ faulty nodes.

BLS Boneh–Lynn–Shacham pairing-based signature scheme.

Calldata Read-only input data of a transaction or call.

CEI Checks-Effects-Interactions: secure Solidity pattern.

ECDLP Elliptic Curve Discrete Logarithm Problem.

ECDSA Elliptic Curve Digital Signature Algorithm.

EOA Externally Owned Account; controlled by a private key.

EVM Ethereum Virtual Machine.

Gas Unit of EVM computation.

GST Global Stabilisation Time.

LMD GHOST Latest Message Driven Greedy Heaviest Observed SubTree.

Merkle tree Binary hash tree enabling $O(\log n)$ membership proofs.

MPT Merkle Patricia Trie; Ethereum's state storage.

NFT Non-Fungible Token (ERC-721).

PoS Proof of Stake.

PoW Proof of Work.

QC Quorum Certificate: $\geq \frac{2}{3}n$ votes for same block.

RLP Recursive Length Prefix encoding.

SNARK Succinct Non-Interactive Argument of Knowledge.

UTXO Unspent Transaction Output.

Wei Smallest ETH unit; 10^{18} wei = 1 ETH.

zk-SNARK Zero-Knowledge SNARK.